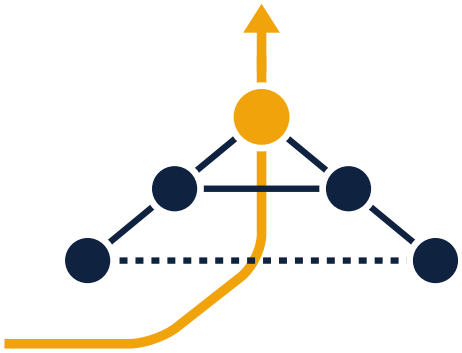
DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING
INDIAN INSTITUTE OF TECHNOLOGY MADRAS
CHENNAI – 600036

PDDL-Based Mission Planning for a Fault-Aware Drone Swarm Simulator



A Thesis

Submitted by

Vadithe Sai Kumar Naik

For the award of the degree

Of

MASTER OF TECHNOLOGY

November 2026

Thesis Certificate

This is to certify that the thesis entitled **PDDL-Based Mission Planning for a Fault-Aware Drone Swarm Simulator**, submitted by **Vadithe Sai Kumar Naik (CS25M053)** to the Indian Institute of Technology Madras in partial fulfilment of the requirements for the degree of **Master of Technology**, is a bona fide record of the work carried out under the guidance of **Dr. N. S. Narayanaswamy**, Project Advisor, and **Dr. Jaimandeep Singh**, Project Co-Advisor.

The contents of this thesis, in full or in part, have not been submitted to any other institute or university for the award of a degree or diploma.

Chennai 600036
Date: November 2026

Vadithe Sai Kumar Naik
CS25M053

Dr. N. S. Narayanaswamy
Project Advisor, Professor
Department of Computer Science and Engineering
Indian Institute of Technology Madras

Dr. Jaimandeep Singh
Project Co-Advisor
Department of Computer Science and Engineering
Indian Institute of Technology Madras

© 2026 Indian Institute of Technology Madras

Acknowledgements

I express my sincere gratitude to **Dr. N. S. Narayanaswamy**, Project Advisor, and **Dr. Jaimandeep Singh**, Project Co-Advisor, for their guidance, feedback and encouragement throughout this project. Their insistence on clear models, careful testing and honest reporting of limitations shaped the way the planning work in this thesis was designed and verified.

I thank the Department of Computer Science and Engineering, Indian Institute of Technology Madras, for the facilities and academic environment that supported this work. I also thank my project teammates, who built and maintained the other parts of the drone swarm simulator, for many useful discussions on how the planner should connect to the ground station and to the flight software.

Finally, I thank my family and friends for their constant support during the project and the preparation of this report. The responsibility for the analysis and conclusions presented here rests with the author.

Abstract

Keywords: automated planning; PDDL; numeric planning; ENHSP; drone swarm; formation flight; fault handling; battery management.

A drone swarm mission is more than a route. Before a drone may take off, join a formation or continue a leg, its battery must cover the next move, its GPS and radio links must work, its health must be acceptable and its neighbours in the formation must be in place. This thesis describes the planning layer of a software-in-the-loop drone swarm simulator, in which these rules are written in the Planning Domain Definition Language (PDDL) and solved by the Expressive Numeric Heuristic Search Planner (ENHSP). The complete PDDL layer is the author's contribution to the project.

Ten mission plans were developed, each as a domain file and a problem template: point-to-point travel, forest area search, V formation, two-dimensional grid formation, three-dimensional (cube) grid formation, parabola formation, figure-eight train formation, star drone show, planetary orbit show and road inspection. Every plan shares one vocabulary of drone states, health flags, failure states and numeric fluents for battery, energy, altitude and health. A plan service retargets each template onto the points the operator picks on the map, expands it to the number of selected drones and passes it to ENHSP. Only the leader corridor is usually planned symbolically; the other drones' routes are then derived geometrically from their formation slots.

Inter-drone communication is modelled in two layers. In PDDL, each formation declares which drones are neighbours and every formation action checks a per-drone `neighbor-comm-ok` flag; the parabola plan goes further and requires each directed link to be established within a communication range. At run time the ground station talks to each drone over a JSON-over-UDP protocol with heartbeats, and formation keeping for trains and shows runs on board each drone against a shared, time-stamped pace. Four fault types are modelled in the domains – GPS loss, communication loss, neighbour-link loss and health failure – together with ground-controller responses such as return, hover and sacrifice. Battery health is handled both as a planning resource (energy per metre, climb and leg energy, a 30-unit reserve) and at run time through low (20 %) and critical (5 %) thresholds that pause the swarm and ask the operator to decide.

The plans were exercised through the real planning pipeline. On the run logs, the forest search solved 69 of 69 requests and point-to-point travel 76 of 83, with all failures traced to infeasible map inputs. The cube formation solves 64 drones in 50 s and the figure-eight train solves up to its computed capacity of 27 drones while correctly rejecting 28. The report also records the modelling defects that were found and fixed, and the limits of the present approach.

Contents

Page

ACKNOWLEDGEMENTS **i**

ABSTRACT **ii**
LIST OF TABLES **vi**

LIST OF FIGURES **vii**

ABBREVIATIONS **viii**

CHAPTER 1 INTRODUCTION **1**

1.1 Background and Motivation 1
1.2 Problem Statement and Objectives 1
1.3 Scope and Contributions 2
1.4 Organisation of the Report 2

CHAPTER 2 BACKGROUND AND LITERATURE REVIEW **3**

2.1 Automated Planning 3
2.2 The Planning Domain Definition Language 3
2.3 Numeric Planning with ENHSP. 3
2.4 Multi-Drone Missions and Formations 4
2.5 Fault-Aware Planning for UAVs. 4
2.6 Position of This Work 4

CHAPTER 3 SYSTEM ARCHITECTURE AND PLANNING PIPELINE **5**

3.1 Simulator Overview 5
3.2 From Map Clicks to Flying Drones 5
3.3 Plan Kinds and Folders. 6
3.4 Run Records. 7

CHAPTER 4 PDDL MODELLING APPROACH **8**

4.1 A Shared Vocabulary 8
4.2 The Common Mission Life Cycle 9
4.3 Design Patterns for Tractable Swarm Models 9

CHAPTER 5 THE MISSION PLANS **11**

5.1 Point-to-Point Travel 11
5.2 Forest Area Search. 12
5.3 V Formation. 12

5.4 Two-Dimensional Grid Formation . . . 12
5.5 Three-Dimensional Cube Formation . . . 13
5.6 Parabola Formation . . . 13
5.7 Figure-Eight Train Formation . . . 14
5.8 Star Drone Show . . . 15
5.9 Planetary Orbit Show . . . 15
5.10 Road Inspection . . . 16
5.11 Summary of the Plans . . . 16

CHAPTER 6 INTER-DRONE COMMUNICATION 18

6.1 Communication in the PDDL Models . . . 18
6.2 The Range-Checked Link Model of the Parabola Plan . . . 19
6.3 Run-Time Communication between Ground Station and Drones . . . 19
6.4 On-Board Formation Keeping . . . 20
6.5 How the Two Levels Relate . . . 21

CHAPTER 7 FAULT MODELLING AND BATTERY HEALTH 22

7.1 Faults Modelled in PDDL . . . 22
7.2 Ground Controller Responses . . . 23
7.3 Battery as a Planning Resource . . . 23
7.4 Battery Health at Run Time . . . 24
7.5 Fault Injection and Operator Decisions . . . 24

CHAPTER 8 VERIFICATION AND RESULTS 26

8.1 Method . . . 26
8.2 In-App Planning Requests . . . 26
8.3 Scalability of the Cube Formation . . . 26
8.4 Figure-Eight Capacity and Flight Quality . . . 27
8.5 Other Plans . . . 27
8.6 A Complete Sample Plan . . . 28
8.7 Defects Found and Fixed . . . 29

CHAPTER 9 DISCUSSION AND LIMITATIONS 30

9.1 What the Models Guarantee . . . 30
9.2 Limitations . . . 30

CHAPTER 10 CONCLUSION AND FUTURE WORK 31

10.1 Conclusion . . . 31
10.2 Future Work . . . 31

REFERENCES 32

APPENDIX A COMPLETE PDDL SOURCES **34**

A.1 Point-to-Point Travel: Domain 34
A.2 Point-to-Point Travel: Problem Template 39
A.3 Parabola Formation: Domain 40
A.4 Parabola Formation: Problem Template 47

List of Tables

Table	Caption	Page
3.1	Components of the simulator and their role in planning.	5
3.2	Inventory of the ten mission plans.	6
4.1	Shared predicates and numeric functions used across the domains	8
5.1	Comparison of the mission plans	17
6.1	Neighbour relations declared by each formation	18
6.2	Commands of the ground-station-to-drone protocol	20
7.1	Fault detection actions shared by the domains	22
7.2	Faults: how each is modelled in PDDL and handled at run time.	25
8.1	Logged planning requests per plan	26
8.2	Cube formation: ENHSP results through the real pipeline	27
8.3	Figure-eight train: ENHSP results on an eight of capacity 27	27
8.4	Verification summary of the remaining plans	28
8.5	Modelling defects found during verification	29

List of Figures

Figure	Caption	Page
3.1	The planning pipeline, from the operator's map input to the drones' flight..	6
4.1	The life cycle shared by the formation plans. Each box is one PDDL action (or one action per drone)..	9
5.1	Rigid formations. The amber drone is the leader whose corridor ENHSP plans; lines show the neighbour links each drone must keep. In the cube, vertical layers are separated by height.	13
5.2	The parabola formation seen from behind. All three pairs keep a direct link (full mesh), and every link is within the 100 m communication range..	14
5.3	The figure-eight train. The leader (amber) and followers fly the same path one spacing apart; the base leg joins the eight tangentially at the crossing..	15
5.4	Shape-changing shows. Both are flown against a shared timetable so that every phase starts and ends together for all drones.	16
6.1	Communication topologies. (a)–(c) are drone-to-drone relations declared in PDDL; (d) is the ground-controller link represented by `communication-ok`..	18
6.2	Run-time messaging. Commands are routed to a port derived from each drone's SYSID; every drone reports back to one ground station socket.	19
6.3	On-board formation keeping. Only the pace is sent from the ground; each drone closes its own gap every tick.	21
7.1	Fault handling in the point-to-point domain: detection, notification and the three controller responses.	23
7.2	Battery thresholds. Below the bar: run-time health classes (C = CRITICAL; FAILED at 0 % in flight). The planner keeps a 30-unit reserve; at 20 % the run time holds the swarm and asks the operator.	24
8.1	Cube formation scalability. Left: planning time (log scale). Right: plan length, close to linear in the number of drones (dashed line $\approx 4.1N + 8$).	27

Abbreviations

Abbreviation	Meaning
PDDL	Planning Domain Definition Language
ENHSP	Expressive Numeric Heuristic Search Planner
GCS	Ground Control Station
GUI	Graphical User Interface
UAV	Unmanned Aerial Vehicle
SYSID	MAVLink system identifier of a drone
MAVLink	Micro Air Vehicle Link, a lightweight drone messaging protocol
SITL	Software in the Loop
UDP	User Datagram Protocol
JSON	JavaScript Object Notation
OSRM	Open Source Routing Machine
AIBR	Additive Interval Based Relaxation (ENHSP preprocessing)
RTL	Return to Launch

Chapter 1

Introduction

1.1 Background and Motivation

Small unmanned aerial vehicles are now used for search, inspection, mapping and public light shows. Many of these tasks are done better by a group of drones than by one drone. A group, however, needs coordination. Each drone must know when to take off, which place in the formation it must hold, which other drones it must stay in contact with, and what to do when something goes wrong. Writing these rules directly in flight code makes them hard to read, hard to check and hard to change.

Automated planning offers a cleaner way. In planning, the rules of the world are written once as a *domain*: a set of actions, each with the conditions under which it may be applied and the changes it causes. A particular mission is written as a *problem*: the drones, the places, the starting state and the goal. A general-purpose planner then searches for a sequence of actions that reaches the goal while respecting every rule. PDDL is the standard language for writing such domains and problems [1, 2].

This project is part of a larger drone swarm simulator built in the department. The simulator runs each drone as an independent thread (and, optionally, real flight firmware under emulation), and offers a ground station GUI with a map, telemetry and fault injection. The work reported here is the planning layer of that simulator: the PDDL models for every mission type, and their link to the rest of the system.

1.2 Problem Statement and Objectives

The central problem is to express swarm missions – including formations, communication requirements, battery limits and failures – in PDDL in a way that a numeric planner can solve quickly for realistic swarm sizes, and whose plans can be turned into flyable routes. The work had the following objectives:

1. Design a common PDDL vocabulary for drone state, health, communication, battery and failure that every mission plan can share.
2. Build domain and problem files for a range of missions: travel, area search, rigid formations (V, grid, cube, parabola), path-following trains (figure-eight), shape-changing shows (star, planets) and road inspection.
3. Model drone-to-drone communication, so that a formation can only form and fly while the required links are up.

4. Model faults (GPS, communication, neighbour link, health) and battery health, together with the ground controller's responses.
5. Integrate the plans with the simulator, so that a plan is retargeted to the operator's map inputs, solved by ENHSP and executed by the drones, and verify the results.

1.3 Scope and Contributions

The author's contribution is the complete PDDL layer of the simulator. This thesis contributes:

- ten mission plans (Table 3.2), about 6 700 lines of PDDL in total, written on one shared vocabulary of predicates and numeric fluents;
- a set of modelling patterns that keep formation problems tractable for large swarms: planning only the leader corridor, descriptive neighbour relations with per-drone communication flags, and propositional rather than numeric gates for ordered launches;
- a two-level model of communication: symbolic neighbour links in PDDL, including an explicit range-checked link model in the parabola plan, and the run-time protocol used by the ground station and the drones;
- a fault and battery model covering four failure types, ground-controller responses and energy accounting for travel, climb, cruise and hover;
- verification of every plan through the real planning pipeline, including scalability measurements and a record of the modelling defects found and fixed.

The GUI, the drone threads, the firmware emulation and the map services were built by the project team. They are described here only as far as needed to explain how a PDDL plan becomes a flight. Physical flight tests are outside the scope of this report; all results are from the simulator.

1.4 Organisation of the Report

Chapter 2 reviews automated planning, PDDL, numeric planning and related work on multi-drone missions. Chapter 3 explains the simulator architecture and the planning pipeline. Chapter 4 presents the common PDDL vocabulary and design patterns. Chapter 5 describes all ten mission plans. Chapter 6 explains how drones communicate, both in the model and at run time. Chapter 7 covers fault modelling and battery health. Chapter 8 reports the verification results and the defects found. Chapter 9 discusses limitations, and Chapter 10 concludes with future work. Appendix A lists complete PDDL sources for two of the plans.

Chapter 2

Background and Literature Review

2.1 Automated Planning

Classical planning studies how an agent can reach a goal from a known initial state by applying actions with known effects [3]. A planning task is described by a set of state variables, a set of actions and a goal. Each action has a *precondition* that must hold before it is applied and an *effect* that changes the state. A *plan* is a sequence of actions that, applied in order from the initial state, ends in a state that satisfies the goal. Heuristic search planners find such plans by estimating, for each state, how far it is from the goal and expanding the most promising states first.

2.2 The Planning Domain Definition Language

PDDL was introduced to give planning competitions a common input language [1]. A PDDL model is split into two files:

- the **domain** defines object types, *predicates* (true/false facts such as `(gps-ok ?d)`), *functions* (numeric values such as `(battery ?d)`) and *actions*;
- the **problem** names the concrete objects, gives the initial facts and values, and states the goal.

PDDL 2.1 added numeric fluents, so that actions can test and change numbers, for example `(> (battery ?d) (energy-required ?from ?to))` as a precondition and `(decrease (battery ?d) ...)` as an effect [2]. Later requirements such as `:conditional-effects`, `:universal-preconditions` and `:negative-precondit` allow an action to say "for every wing drone, …" or "only if this drone is *not* already in formation". All of these features are used in this work.

2.3 Numeric Planning with ENHSP

ENHSP is a heuristic search planner for PDDL 2.1 problems with numeric fluents [4, 5]. It grounds the domain into concrete actions, runs a relaxation-based preprocessing step (AIBR) that also detects some unsolvable problems early, and then searches with numeric heuristics. Two properties of ENHSP mattered in this project. First, its heuristics estimate progress well for conditions that compare a fluent with a constant, but poorly for conditions that compare two fluents with each other. Second, its grounding step can use a large amount of memory for domains with many drones and quantified conditions; a "naive" grounder option (`-gro naive`) helps for formation domains. Both points shaped

the models in Chapter 4.

2.4 Multi-Drone Missions and Formations

Aerial swarm research covers formation control, task allocation and collective behaviour [6, 7]. Formation flight is often treated with a leader–follower scheme, in which one drone follows the planned path and the others hold an offset from it. Early work on flocking showed that simple local rules with near neighbours are enough for coherent group motion [8]. This project borrows both ideas: the planner reasons about a leader corridor, and each drone is required to stay in contact only with its immediate neighbours. For curved paths, shortest paths with a bounded turning radius were characterised by Dubins [9], and such paths are used to join the figure-eight pattern smoothly.

2.5 Fault-Aware Planning for UAVs

Real drones lose GPS, lose radio contact and run out of battery. A plan that ignores these events is not safe to fly. In planning, faults are commonly handled either by monitoring execution and replanning, or by including recovery actions in the domain. This work follows the second idea in a light form: each domain includes detection actions that turn a failed health flag into a failure state and a controller notification, and the point-to-point domain adds explicit controller orders (return, hover, sacrifice). Battery is treated as a numeric resource, so that the planner itself refuses any move the drone cannot afford.

2.6 Position of This Work

The contribution is not a new planning algorithm. It is a family of carefully engineered PDDL models for drone swarm missions, connected to a working simulator, together with an honest record of what the models express, how they scale and where they stop. The models are written to be read by people as well as solved by a planner, so that a supervisor or teammate can check every safety rule directly in the PDDL.

Chapter 3

System Architecture and Planning Pipeline

3.1 Simulator Overview

The simulator has four main parts. The **ground station GUI** (PySide6) shows the map, telemetry, the mission planner panel and fault injection controls. The **plan service** prepares PDDL problems and calls the planner. The **planner** is ENHSP, run as a Java process. The **drones** are independent threads, each with its own state, command inbox and telemetry output; a drone can also be backed by real flight firmware running under Renode emulation. Table 3.1 summarises the roles.

Table 3.1: Components of the simulator and their role in planning

Component	Responsibility
Mission planner panel (GUI)	Choose a named plan, pick source, destination, area or pattern points on the map, set laps and show speed.
Plan service (`services/plan_service.py`)	Select the plan folder, retarget the template to the picked points, expand it to *N* drones, call ENHSP, check the returned action order and build per-drone routes.
Problem tools (`engine/pddl_problem.py` and per-plan modules)	Rewrite coordinates, distances and energy facts; clone template drones; build neighbour chains and geometry.
ENHSP (`engine/pddl_planner.py`)	Ground and solve the problem; return the plan text, which is parsed into ordered steps.
Thread backend and GCS (`services/thread_backend.py`, `engine/gcs.py`)	Send missions and commands to each drone, receive telemetry and heartbeats, keep formation pace and raise low-battery, GPS-loss and drone-lost events.
Drone thread (`engine/drone_thread.py`)	Integrate motion and battery, monitor health and communication, fly the uploaded route.

3.2 From Map Clicks to Flying Drones

Figure 3.1 shows the planning pipeline. Every plan folder under `plans/` holds one `domain.pddl` and one problem template. The template is written for a small, fixed example (for example one leader and two wings). When the operator picks points on the map, the plan service *retargets* the template: it replaces the coordinates, recomputes each edge's distance and `energy-required` at that edge's own energy-per-metre rate, and, if a restricted area was marked, inserts detour locations. It then *expands*

the template to the number of selected drones by cloning the template drone's facts line by line and rebuilding the neighbour relations. The resulting problem is passed to ENHSP. The returned plan is checked against the order of actions the plan kind expects, and the leader's corridor is extracted. Finally, member routes are derived from the corridor and the slot offsets, and the routes are uploaded to the drones as whole missions.

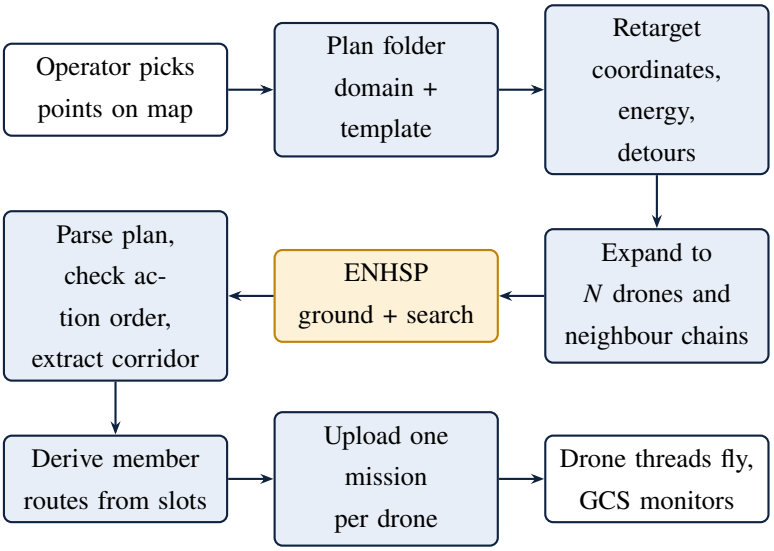


Figure 3.1: The planning pipeline, from the operator's map input to the drones' flight.

3.3 Plan Kinds and Folders

The plan service decides how to treat a plan from the *domain name* written inside `domain.pddl`, not from the folder name. This lets a folder be renamed freely and makes the domain file the single source of truth. Table 3.2 lists all ten plans.

Table 3.2: Inventory of the ten mission plans

Folder	Domain name	**Kind**	**Drones**
`travell`	`swarm-drone-mission`	point_to_point	1 or more
`Search`	`forest-drone-search`	area_coverage	≥ 2
`vformation`	`v-formation-drone-mission`	formation	odd, ≥ 3
`gridformation`	`grid-formation-drone-mission`	grid_formation	≥ 4
`3dgridformation`	`cube-grid-formation-drone-mission`	cube_formation	≥ 8
`parabolaformation`	`parabola-formation-drone-mission`	parabola_formation	exactly 3
`figure8formation`	`figure8-formation-drone-mission`	figure8_formation	2 to capacity
`starformation`	`star-formation-drone-mission`	star_formation	6 to 40
`planets`	`planets-formation-drone-mission`	planets_formation	2 to 8
`Inspect`	`road-inspection-drone-mission`	road_inspection	1

3.4 Run Records

Every planning request writes a time-stamped folder under `plans/<name>/runs/` containing the exact problem that was solved and the full ENHSP output. These records made it possible to trace every unsolvable request back to its input, and they are the source of the statistics in Chapter 8.

Chapter 4

PDDL Modelling Approach

4.1 A Shared Vocabulary

All ten domains use the same core vocabulary, so that a reader who understands one plan can read any other. The domains declare the types `drone`, `location` and `controller` (plus plan-specific types such as `forest-area` or `layer`). Table 4.1 lists the shared predicates and functions.

Table 4.1: Shared predicates and numeric functions used across the domains

Symbol	**Meaning**
Position and flight state	
`(at ?d ?l)`, `(formation-at ?l)`	Where a drone, or the whole formation, is.
`(landed ?d)`, `(airborne ?d)`, `(flying ?d)`	Flight phase of a drone.
`(at-cruise-altitude ?d)`	Drone has climbed to its own target altitude.
`(connected ?a ?b)`, `(safe-route ?a ?b)`	A leg exists and avoids restricted areas.
Health and communication	
`(gps-ok ?d)`, `(communication-ok ?d)`	GPS fix and link to the ground controller are working.
`(drone-healthy ?d)`	Airframe passes its health check.
`(neighbor-comm-ok ?d)`	Drone-to-drone links to its formation neighbours are working.
`(gps-lost ?d)`, `(communication-lost ?d)`, `(health-failure ?d)`	Failure states set by detection actions.
`(controller-notified ?d)`	Ground controller has been told of a failure.
Numeric fluents	
`(battery ?d)`, `(max-battery ?d)`	Current and full battery, in model units (100 = full).
`(minimum-return-battery ?d)`	Reserve kept for a safe return (30 in the templates).
`(energy-required ?a ?b)`, `(distance ?a ?b)`	Cost and length of a leg.
`(climb-energy ?d)`, `(cruise-leg-energy ?d)`	Energy for the climb and for one follower leg.
`(health ?d)`, `(minimum-health ?d)`	Health score (100) and its limit (40).
`(altitude ?d)`, `(hover-altitude ?d)`, `(target-altitude ?d)`	Current, hover (2 m) and cruise altitude.
`(slot-along-offset ?d)`, `(slot-cross-offset ?d)`	Formation slot relative to the leader, in metres.

4.2 The Common Mission Life Cycle

Every formation plan follows the same life cycle (Figure 4.1). A drone first passes a `pre-flight-check`, which needs it to be landed at the source with GPS, radio and health in order and a battery above its return reserve. It then takes off to hover altitude and climbs to its own cruise altitude, paying `climb-energy`. Once airborne, it takes its slot in the formation. An `establish-...-formation` action fires only when every member is in its slot, at altitude and in contact with its neighbours. The leader then flies the corridor one leg at a time with `formation-cruise`; after each leg the formation is marked "open" and a close-up action (`close-up-wings`, `close-up-grid`, `hold-parabola-shape`, `follow-leader`) must bring every member back into its slot before the next leg. The mission completes when the closed formation is at the destination.

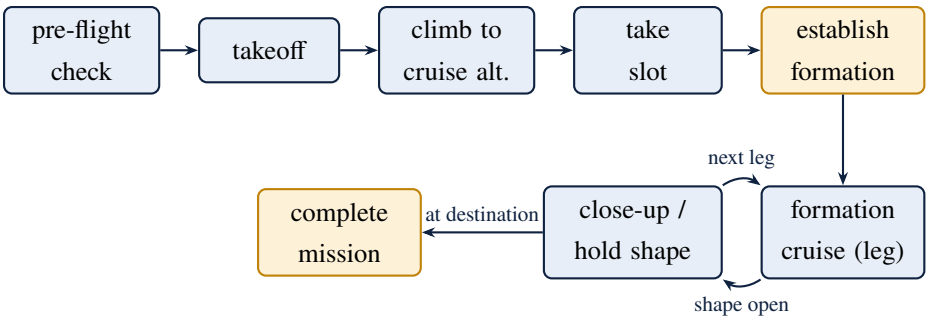


Figure 4.1: The life cycle shared by the formation plans. Each box is one PDDL action (or one action per drone).

The take-off step itself is short but illustrates the style used throughout:

```
(:action takeoff
    :parameters (?d - drone ?l - location)
    :precondition (and (at ?d ?l) (source ?l) (landed ?d)
                       (preflight-done ?d)
                       (gps-ok ?d) (communication-ok ?d) (drone-healthy ?d))
    :effect (and (not (landed ?d)) (airborne ?d) (flying ?d)
                 (assign (altitude ?d) (hover-altitude ?d))))
```

4.3 Design Patterns for Tractable Swarm Models

Swarm problems grow quickly with the number of drones. Four patterns kept the models small enough for ENHSP to solve within seconds.

Plan the leader, derive the rest. For rigid formations the planner reasons about the leader's corridor only. Members are moved together by one close-up action with a universally quantified, conditional effect, instead of being routed one by one. Their geographic routes are computed afterwards by offsetting the corridor by the slot offsets. This keeps the number of ground actions close to linear in N.

Descriptive neighbours, per-drone flags. Each formation records which drones are neighbours (`wing-neighbor`, `grid-neighbor`, `cube-neighbor`, `train-neighbor`, `orbit-neighbor`). In most plans these pairwise facts are descriptive only: the actions test the per-drone flag `neighbor-comm-ok` instead. A check over all drones is then $O(N)$ rather than $O(N^2)$, which matters most for the cube, where a full $4 \times 4 \times 4$ block declares 420 undirected links.

Propositional gates instead of numeric comparisons. The cube's layer-by-layer launch was first written with a numeric counter, gating take-off on `(<= (slot-layer ?d) (layers-cleared))`. ENHSP's heuristic estimates almost no progress for a comparison between two fluents, so a 25-drone, 3-layer block did not solve in 90 s. The gate was rewritten with `layer` objects chained by `(layer-below ?lo ?hi)` and the flags `layer-launched` and `cleared-to-launch`. The same problem then solved in 1.2 s.

One fact per line in templates. Templates are expanded by cloning facts line by line. Every per-drone fact must therefore sit on a single line, for example `(= (hover-altitude drone-member) 2)`. A multi-line fact was the root cause of the grid formation defect described in Section 8.7.

Chapter 5

The Mission Plans

This chapter describes each of the ten plans: what the mission does, how it is picked on the map, the key facts of its model and the ordering of its actions.

5.1 Point-to-Point Travel

Mission. The whole selected swarm flies one shared route from a picked source to a picked destination, optionally around a marked no-fly area. The template's single drone `drone1` is cloned to every selected drone.

Model. This is the base domain, and it contains the richest fault handling (Chapter 7). Before each move a drone performs `check-distance` and `check-battery`; if the battery cannot cover the leg, `notify-insufficient-battery` records the fact instead. A route is planned only when every safety check passes:

```
(:action plan-safe-route
    :parameters (?d - drone ?from - location ?to - location)
    :precondition (and (at ?d ?from) (connected ?from ?to) (safe-route ?from ?to)
                       (battery-checked ?d)
                       (gps-ok ?d) (communication-ok ?d) (drone-healthy ?d))
    :effect (route-planned ?d))
```

The `travel` action moves the drone, sets it flying, decreases its battery by `energy-required` and clears `route-planned`, so the checks are repeated for every leg. A restricted direct route is marked with `restricted-route` and has no `safe-route` fact, so the planner must use the detour waypoints. A typical three-drone plan from the run logs is shown below.

ENHSP output — travel plan, three drones (run 2026-10-12_091304)

```
0.0:  (check-distance drone3 source destination)
3.0:  (check-battery drone1 source waypoint1)
6.0:  (plan-safe-route drone3 source destination)
9.0:  (travel drone2 source destination)
12.0: (reach-destination drone2 destination)
...   Plan-Length: 15   Planning Time: 34 ms
```

5.2 Forest Area Search

Mission. A marked forest polygon is split into lanes, and each drone covers its own lane from a common base and returns. Unlike travel, each drone has its own route. At least two drones are needed.

Model. The type `forest-area` represents a lane, and `(assigned ?d ?a)` gives each drone its lane. The domain adds an explicit separation check between any two drones, using a pairwise numeric limit of 10 m:

```
(:action check-drone-separation
    :parameters (?d1 - drone ?d2 - drone ?l1 - location ?l2 - location)
    :precondition (and (at ?d1 ?l1) (at ?d2 ?l2) (not (= ?d1 ?d2))
                       (>= (distance ?l1 ?l2) (minimum-drone-separation ?d1 ?d2)))
    :effect (and (safe-separation ?d1 ?d2) (separation-ok ?d1) (separation-ok ?d2)))
```

The action sequence is `start-search`, `move-search` and `cover-area` per lane, `complete-forest-search`, `return-to-base` and `complete-drone-mission`. An `emergency-low-battery-return` action marks a drone as returning whenever its battery falls to the return reserve.

5.3 V Formation

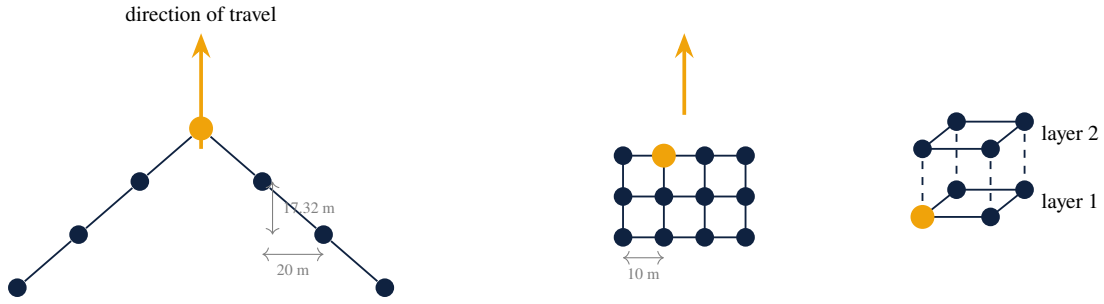
Mission. Picked like travel. A leader (apex) flies the corridor and the other drones form two wings behind it. Each rank sits 17.32 m back and 20 m to the side of the one before it, so the wing angle is 60° (Figure 5.1a). Extra drones add further ranks.

Model. Wings launch after a short delay: `hold-position` lets the airborne leader wait, increasing `seconds-since-leader-airborne` until it reaches `wing-launch-delay`. Each wing drone takes its slot, and `establish-v-formation` requires every wing drone to be in formation, at the same location, at altitude and with `neighbor-comm-ok`. The pairwise fact `(wing-neighbor ?d1 ?d2)` records the chain leader–rank 1–rank 2 along each wing; the expansion code rebuilds this chain correctly for any larger odd drone count.

5.4 Two-Dimensional Grid Formation

Mission. The drones fly as a flat grid, 10 m apart in every direction (Figure 5.1b). A square count gives a square grid; otherwise the most square rectangle that tiles the count exactly is used. At least four drones are needed.

Model. Each drone coordinates only with up to four neighbours (front, back, left, right), recorded by `grid-neighbor`. The leader's corridor is planned, and `close-up-grid` restores every member's cell after each leg. This domain needs ENHSP's naive grounder beyond about a dozen drones.



(a) V formation (b) 3×4 grid (c) $2 \times 2 \times 2$ cube

Figure 5.1: Rigid formations. The amber drone is the leader whose corridor ENHSP plans; lines show the neighbour links each drone must keep. In the cube, vertical layers are separated by height.

5.5 Three-Dimensional Cube Formation

Mission. The grid extended into the vertical: a solid block of drones flies one corridor, 10 m apart horizontally and vertically (Figure 5.1c). A square footprint is chosen and as many layers as needed are stacked, allowing a partly filled top layer (for example 9 drones become $3 \times 2 \times 2$ with 9 of 12 cells filled; 25 become $3 \times 3 \times 3$). At least eight drones are needed.

Model. Layers are separated by height, so each drone has its own `target-altitude` (base plus 10 m per layer), and the run-time route adds a per-drone altitude offset. The swarm climbs layer by layer, bottom first:

```
(:action open-next-layer
    :parameters (?lo - layer ?hi - layer)
    :precondition (and (layer-below ?lo ?hi) (layer-launched ?lo)
                       (not (layer-launched ?hi))
                       (forall (?d - drone)
                           (or (not (in-layer ?d ?lo)) (at-cruise-altitude ?d))))
    :effect (and (layer-launched ?hi)
                 (forall (?d - drone)
                     (when (in-layer ?d ?hi) (cleared-to-launch ?d)))))
```

Each drone may coordinate with up to 24 neighbours: the $3 \times 3 \times 3$ block around it, except itself and the cells directly above and below. Collision avoidance is structural: fixed spacing, layer-by-layer ascent and a `cube-closed` interlock that must hold before each leg.

5.6 Parabola Formation

Mission. Three drones fly side by side on one line across the route, 10 m apart. The middle drone (the vertex) cruises at 50 m and the two arm drones at 60 m, so seen from behind they form an upward-opening parabola (Figure 5.2). Extra selected drones stay on the ground.

Model. This plan has the most detailed communication model (Section 6.2). Before any drone may take its slot, every directed link must be established and each drone must confirm it is in contact with

all of its neighbours. The formation action checks the shape: each arm's target altitude minus the vertex's must equal `arm-rise` (10 m), so a flat line, an inverted U or lopsided arms give no plan, and every vertex–arm gap (14.14 m) must be at least `min-separation` (10 m).

```
(:action establish-parabola-formation
  :parameters (?vtx - drone ?l - location)
  :precondition (and (formation-leader ?vtx) (slot-vertex ?vtx) (in-formation ?vtx)
     (at ?vtx ?l) (at-cruise-altitude ?vtx) (swarm-comm-ready ?vtx)
     (forall (?a - drone)
       (or (not (arm-drone ?a))
           (and (in-formation ?a) (at ?a ?l) (at-cruise-altitude ?a)
                (= (- (target-altitude ?a) (target-altitude ?vtx)) (arm-rise))
                (>= (link-distance ?vtx ?a) (min-separation))
                (swarm-comm-ready ?a)))))
  :effect (and (parabola-established) (formation-at ?l) (shape-held)))
```

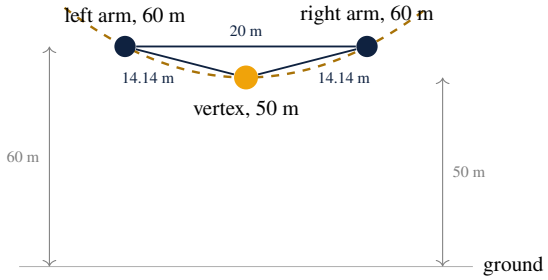


Figure 5.2: The parabola formation seen from behind. All three pairs keep a direct link (full mesh), and every link is within the 100 m communication range.

At run time the vertex corridor is split into legs of at most 200 m, and the arms are derived parallel at ±10 m. All three drones fly one shared timetable, so they arrive at each waypoint together even though the vertex has 10 m less to climb.

5.7 Figure-Eight Train Formation

Mission. The swarm flies a figure-eight in the air as a *train*: one drone behind another along the curve, 10 m apart. The operator makes four clicks: a base, then the left end, the crossing point and the right end of the eight. Each lobe is half a lemniscate of Gerono from the crossing to its clicked end, so the lobes cross at right angles and may differ in size (Figure 5.3). The drones fly from the base to the crossing along a Dubins path (turn radius at least 25 m), fly the requested number of laps, and return to their own landing pads.

Model. Each drone talks only to the drone directly in front and behind (`train-neighbor`). The leader's lap is planned through nine key waypoints with `formation-cruise`, and `follow-leader` moves the followers after every leg. Laps are counted numerically:

```
(:action start-next-lap
  :parameters (?lead - drone ?dest - location ?start - location)
  :precondition (and (lap-end ?dest) (lap-start ?start) (figure8-established)
                     (train-closed) (formation-at ?dest) (train-leader ?lead)
                     (at ?lead ?dest) (< (+ (laps-completed) 1) (laps-required)))
```

```
:effect (and (not (formation-at ?dest)) (formation-at ?start)
             (not (at ?lead ?dest)) (at ?lead ?start)
             (increase (laps-completed) 1)))
```

Capacity. The only place the train can meet itself is the crossing. The train length $(N-1) \times 10\,\text{m}$ plus 14.1 m of crossing clearance must therefore fit inside the smaller lobe. This rule is checked in the GUI before planning, again in the plan worker, and inside PDDL itself as a precondition of `establish-figure8-formation`, so ENHSP refuses an over-full eight.

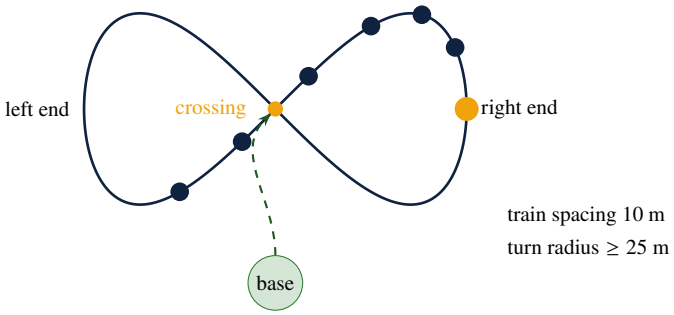


Figure 5.3: The figure-eight train. The leader (amber) and followers fly the same path one spacing apart; the base leg joins the eight tangentially at the crossing.

5.8 Star Drone Show

Mission. A stationary show around one picked point. The drones take off from pads in rings, spread into a circle, and then, for each lap, form a star, return to the circle, spin the circle once, form the star again, spin the star and return to the circle, before landing on their own pads (Figure 5.4a). Six or seven drones make a 3-pointed star, eight or nine a 4-pointed star, and ten or more a 5-pointed star; 6 to 40 drones are allowed.

Model. ENHSP sequences `form-circle`, then per lap `form-star`, `form-circle-in-lap`, `spin-circle`, `form-star-again`, `spin-star` and `close-lap`, and finally `land-show`. The plan worker checks that exactly this sequence is returned. Every drone keeps the same order round the centre in the circle and the star, so shape changes never cross paths; the show is sized until all drones are at least 10 m apart in each shape and at least 9 m apart during transitions.

5.9 Planetary Orbit Show

Mission. A solar-system show round one picked point (the sun), with 2 to 8 drones, one per planet. Drone k orbits at a radius of $50(k+1)$ m, and neighbouring orbits turn in opposite directions (Figure 5.4b). One lap is one turn of the outermost drone; inner drones make as many whole turns as fit, so all drones line up again on the north line at the end of every lap.

Model. `enter-orbits` may fire only when every drone has climbed and has `neighbor-comm-ok`, and only if every pair of orbit neighbours turns in opposite directions:

```
(forall (?a - drone ?b - drone)
    (or (not (orbit-neighbor ?a ?b))
        (and (orbits-clockwise ?a) (orbits-anticlockwise ?b))
        (and (orbits-anticlockwise ?a) (orbits-clockwise ?b))))
```

One `fly-lap` follows per lap, counted by `laps-completed`, then `land-planets`.



(a) star show, 10 drones (b) planets, alternating directions

Figure 5.4: Shape-changing shows. Both are flown against a shared timetable so that every phase starts and ends together for all drones.

5.10 Road Inspection

Mission. The operator clicks a source and a destination on a road. One drone takes off at the source, follows the road's exact path to the destination, flies the same road back and lands at the source. The road geometry comes from an OSRM routing server on OpenStreetMap data [10].

Model. The road is split into at most 12 segments, each with its own length and energy. ENHSP plans `inspect-segment` for each in order, `finish-inspection` at the destination, `return-along-road` back segment by segment, and `land-at-source`. Each segment inspection checks GPS, communication, health and that the battery covers the segment. If no road route exists, a click is more than 30 m from any road, or the two points are less than 20 m apart along the road, the plan fails before anything flies.

5.11 Summary of the Plans

Table 5.1 compares the ten plans.

Table 5.1: Comparison of the mission plans

Plan	Map input	What ENHSP plans	Special feature
Travel	source, destination, no-fly area	every drone's legs	full controller fault responses
Search	base, forest polygon	one lane per drone	pairwise separation check
V	source, destination, no-fly area	leader corridor	timed wing launch
Grid	as V	leader corridor	4-neighbour links
Cube	as V	leader corridor	layer-by-layer ascent
Parabola	as V	vertex corridor	range-checked link mesh, shape check
Figure-8	base and three points	leader's lap	capacity rule, laps
Star	one point	show phases	circle/star/spin sequence
Planets	one point	show phases	opposite-direction orbits
Inspect	two points on a road	road segments	OSRM road geometry

Chapter 6

Inter-Drone Communication

Communication appears at two levels in the simulator. In the PDDL models it is a set of conditions: a drone may only act if its links are up. At run time it is a real exchange of messages between the ground station and the drones. This chapter explains both and how they relate.

6.1 Communication in the PDDL Models

Two flags carry communication state for every drone:

- `communication-ok`: the link between the drone and the ground controller works;
- `neighbor-comm-ok`: the drone's links to its formation neighbours work.

Every action that moves a drone or changes the formation checks the relevant flags. A formation cannot be established, cruise a leg or close up if any member has lost its neighbour links. Each plan also declares *who* must talk to whom, using a neighbour relation suited to its shape (Table 6.1, Figure 6.1).

Table 6.1: Neighbour relations declared by each formation

Plan	Relation	**Who talks to whom**
V formation	`wing-neighbor`	leader with rank 1, then rank to rank along each wing (a chain)
Grid	`grid-neighbor`	up to 4 neighbours: front, back, left, right
Cube	`cube-neighbor`	up to 24 neighbours in the surrounding $3 \times 3 \times 3$ block, excluding directly above and below
Parabola	`parabola-neighbor`	all three pairs, in both directions (6 directed links)
Figure-8	`train-neighbor`	the drone in front and the drone behind
Planets	`orbit-neighbor`	drones on adjacent orbits



(a) chain V wings, train — (b) 4-neighbour grid — (c) full mesh parabola — (d) ground link (all plans)

Figure 6.1: Communication topologies. (a)–(c) are drone-to-drone relations declared in PDDL; (d) is the ground-controller link represented by `communication-ok`.

6.2 The Range-Checked Link Model of the Parabola Plan

In most plans the links are summarised by the per-drone flag. The parabola plan models each directed link explicitly. A link from d_1 to d_2 can be established only when both drones are airborne at cruise altitude, both radios work, and the straight-line distance between them is within the communication range of 100 m:

```
(:action establish-comm-link
    :parameters (?d1 - drone ?d2 - drone)
    :precondition (and (parabola-neighbor ?d1 ?d2) (not (link-up ?d1 ?d2))
                       (airborne ?d1) (airborne ?d2)
                       (at-cruise-altitude ?d1) (at-cruise-altitude ?d2)
                       (communication-ok ?d1) (communication-ok ?d2)
                       (neighbor-comm-ok ?d1) (neighbor-comm-ok ?d2)
                       (<= (link-distance ?d1 ?d2) (comm-range)))
    :effect (link-up ?d1 ?d2))
```

A drone becomes `swarm-comm-ready` through `confirm-swarm-comm` once it has a link to every one of its neighbours (a universally quantified condition). Only then can it take its slot. During flight, `hold-parabola-shape` requires every declared link to still be up, and if a drone loses its neighbour links, `detect-neighbor-link-loss` removes all links to and from that drone, so the shape cannot be re-formed until they are restored. A negative test with the range reduced to 15 m (below the 20 m arm-to-arm distance) correctly produced no plan. In the solved plan, the six `establish-comm-link` actions and three `confirm-swarm-comm` actions appear right after the climbs and before any slot is taken (Section 8.6).

6.3 Run-Time Communication between Ground Station and Drones

At run time each drone is an independent thread that owns its state. It can only be reached through a command inbox and only reports through a telemetry callback. Over the network, the ground station and drones use a simple JSON-over-UDP protocol, one datagram per message, designed so that it can later be replaced by MAVLink [11]. Every message carries the drone's MAVLink SYSID, so one ground socket can address the whole swarm (Figure 6.2).

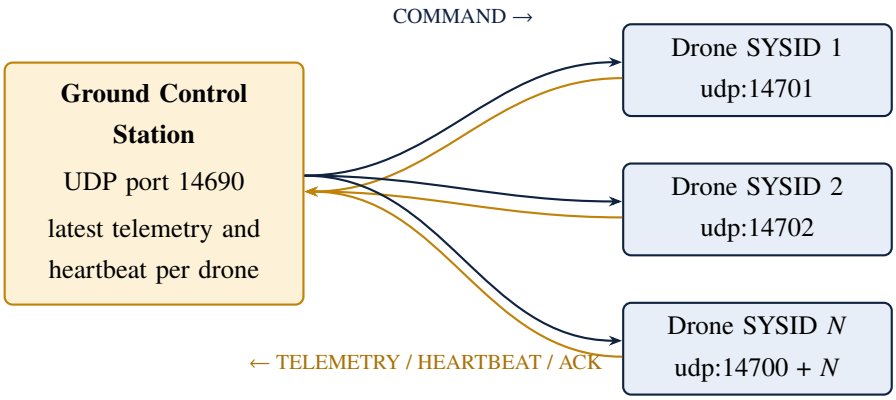


Figure 6.2: Run-time messaging. Commands are routed to a port derived from each drone's SYSID; every drone reports back to one ground station socket.

Table 6.2 lists the commands. A drone sends a heartbeat every second and telemetry every 0.5 s. The ground station treats a drone as out of contact if no heartbeat arrives for 3 s. On the drone side, if nothing is heard from the ground for more than 5 s the drone marks its link `DEGRADED`, and after 10 s `LOST`. Link loss is therefore *observed* from missing messages rather than assumed.

Table 6.2: Commands of the ground-station-to-drone protocol

Command	**Purpose**
`NAVIGATE`	The one mission operation: upload a whole route (approach, loop, laps, depart).
`ARM, DISARM, TAKEOFF, LAND`	Basic flight control.
`HOLD, RESUME`	Pause and continue the mission (used while the operator decides on a fault).
`RETURN_TO_LAUNCH, ABORT`	Fall back to the start point, or stop.
`SET_GPS`	Fault injection: switch the drone's GPS fix off or on.
`SET_SPEED`	Change cruise speed in flight.
`TRAIN_PACE`	Formation keeping: the shared, time-stamped pace of the train or show clock.
`PING, SHUTDOWN`	Liveness check and stop the drone thread.

6.4 On-Board Formation Keeping

Early versions of the figure-eight plan let the ground station correct each drone against its neighbours' live positions. Because speed commands arrived two or three polls late, which is many metres of flight at the simulator's time scale, a train of drones each over-correcting to the one in front began to oscillate, and drones bunched and overtook each other. The fix moved formation keeping on board (Figure 6.3). The ground station now only broadcasts a shared, time-stamped `TRAIN_PACE`. Every tick, each drone works out where it should be (the pace extrapolated from its time stamp, minus its place in the train), measures its own exact progress along its own route, and sets its own autopilot speed to close the gap. Neighbour information is still used, but only as a cap: if the drone in front is reported too close, the drone does not speed up into it; if a drone falls behind, the ground station slows the pace so the train waits.

For the star and planets shows the same mechanism runs in *schedule* mode: each drone gets keyframes "at clock time t, be s metres along your route", and the pace is a shared clock. Every drone flying its own timetable against one clock is what makes every shape change start and finish together.

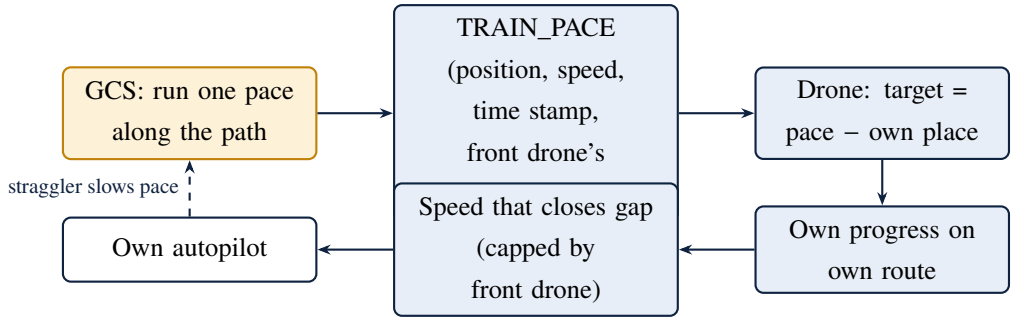


Figure 6.3: On-board formation keeping. Only the pace is sent from the ground; each drone closes its own gap every tick.

6.5 How the Two Levels Relate

The PDDL flags are a *planning-time* statement: the plan is only valid under the assumption that the listed links work. The run-time protocol is what actually carries commands and reports. The two are connected through the fault events of Chapter 7: when the run time observes a fault, the mission is held and the operator decides how to continue. The PDDL conditions do not by themselves monitor real radio links; this distinction is discussed in Chapter 9.

Chapter 7

Fault Modelling and Battery Health

7.1 Faults Modelled in PDDL

Every domain includes the same family of detection actions. Each one fires for a flying drone whose health flag has failed, records a failure state, and takes the drone out of flight and, in formation plans, out of the formation. Table 7.1 lists them.

Table 7.1: Fault detection actions shared by the domains

Action	**Fires when**	**Effect**
`detect-gps-loss`	flying and not `gps-ok`	`gps-lost`; not flying; leaves the formation (travel also sets `gps-warning`)
`detect-communication-loss`	flying and not `communication-ok`	`communication-lost`; not flying; leaves the formation; parabola also clears `swarm-comm-ready`
`detect-neighbor-link-loss`	flying and not `neighbor-comm-ok` (formation plans)	`communication-lost`; leaves the formation; parabola also drops every link to and from the drone
`detect-health-failure`	flying and `health` < `minimum-health` (40)	`health-failure`; not flying; leaves the formation
`notify-ground-controller`	any of the three failure states	`controller-notified`

The GPS loss detection, for example, is:

```
(:action detect-gps-loss
    :parameters (?d - drone)
    :precondition (and (flying ?d) (not (gps-ok ?d)))
    :effect (and (gps-lost ?d) (not (flying ?d)) (not (in-formation ?d))))
```

Because every movement and formation action also requires `gps-ok`, `communication-ok`, `drone-healthy` and, for formations, `neighbor-comm-ok`, a problem that starts with a failed flag has no plan that moves the affected drone. The planner thus refuses to fly an unhealthy drone, and a failed member blocks the formation actions that depend on it.

7.2 Ground Controller Responses

The point-to-point domain goes further and models what the ground controller can order once it has been notified (Figure 7.1):

- **Return**: `controller-order-return` sets `emergency-return`, and `return-to-source` flies the drone back over a safe route, provided the battery covers the return leg.
- **Hover**: `controller-order-hover` puts the drone in a hover. The `hover` action drains `hover-energy` while the battery stays above the return reserve; when it reaches the reserve, `initiate-low-battery-return` switches the drone to an emergency return.
- **Sacrifice**: `sacrifice-drone` lands the drone immediately where it is when the controller decides it cannot be recovered.

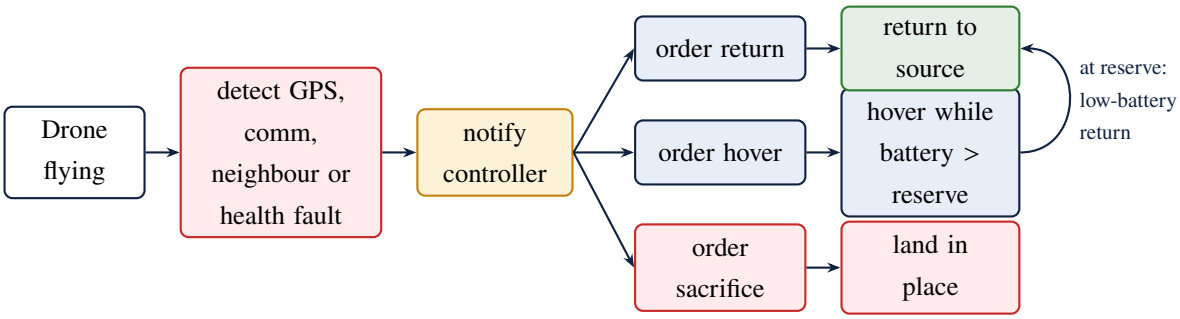


Figure 7.1: Fault handling in the point-to-point domain: detection, notification and the three controller responses.

7.3 Battery as a Planning Resource

Battery is a numeric fluent in every domain, in model units with 100 as a full charge. It is used in three ways.

Energy per leg. Each leg has an `energy-required` value. When a template is retargeted, this value is recomputed from the new leg length at the leg's own energy-per-metre rate, with a default of 0.0125 units per metre (12.5 units per kilometre). Every move requires the battery to be strictly greater than the leg's cost and decreases it by that cost.

Climb, cruise and hover energy. Formation plans charge `climb-energy` for the climb to cruise altitude (in the parabola plan 4 units for the vertex and 5 for the arms, which climb 10 m further), and `cruise-leg-energy` (12 units) for each follower or arm leg. The travel domain charges `hover-energy` for each hover step.

Reserve for return. Every drone keeps a `minimum-return-battery` of 30 units. `pre-flight-check` refuses a drone whose battery is not above this reserve, `check-battery` and `notify-insufficient-battery` classify each leg as affordable or not, and the hover and search domains switch to a return when the

battery reaches the reserve. Health is a separate numeric score (100 initially, minimum 40), so a drone can be refused for poor health even with a full battery.

7.4 Battery Health at Run Time

During flight each drone thread integrates its own battery. With the default settings a drone has a 5 200 mAh battery and draws a nominal 20 A in flight; its percentage is computed from the consumed charge. The drone classifies its own health from the battery and the GPS fix (Figure 7.2): `NOMINAL` above 20 %, `DEGRADED` at or below 20 % or without GPS, `CRITICAL` at or below 5 %, and `FAILED` if the battery is exhausted in flight, in which case the drone is lost.

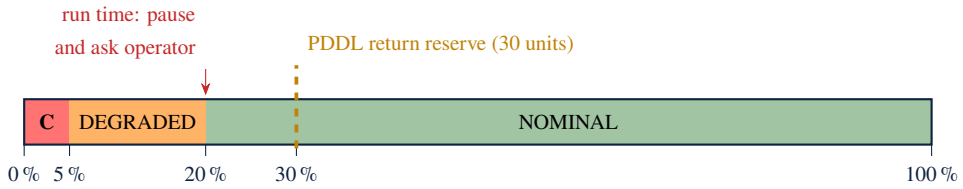


Figure 7.2: Battery thresholds. Below the bar: run-time health classes (C = CRITICAL; FAILED at 0 % in flight). The planner keeps a 30-unit reserve; at 20 % the run time holds the swarm and asks the operator.

When any drone crosses the low-battery threshold, the ground station *pauses the whole mission* so that the drones do not drain while the operator decides; at the compressed time scale the decision window would otherwise be a fraction of a second. A dialog shows the drone's battery and the nearest landing point, with a note whether it is within reach on the remaining charge. The operator may divert and land now, or press on, in which case the drone will be lost when the battery runs flat. A drone whose battery reaches zero in flight is reported as lost and sacrificed.

7.5 Fault Injection and Operator Decisions

The GUI's fault injection panel lets the operator target one or more drones by SYSID with a fault type (GPS loss, communications loss or battery failure), a severity and a duration (zero meaning until reset). In the thread-based backend, GPS loss is delivered as a `SET_GPS` command, so the drone itself loses its fix. On GPS loss the ground station holds the swarm and asks the operator whether the drone should fall back to its start point and land, showing the distance back and whether the remaining charge covers it; otherwise the drone holds position until GPS returns, while its battery keeps draining. When GPS is reacquired the mission resumes. Table 7.2 maps each fault to its PDDL model and run-time behaviour.

Table 7.2: Faults: how each is modelled in PDDL and handled at run time

Fault	PDDL model	Run-time behaviour
GPS loss	not `gps-ok`; `detect-gps-loss`	injected with `SET_GPS`; drone `DEGRADED`; swarm held; operator chooses fall-back or hold
Communication loss	not `communication-ok`; `detect-communication-loss`	observed from missing heartbeats and contact (`DEGRADED` after 5 s, `LOST` after 10 s)
Neighbour link loss	not `neighbor-comm-ok`; `detect-neighbor-link-loss`	reflected in formation keeping through the front-drone cap and pace slowing
Health failure	`health` below `minimum-health`; `detect-health-failure`	drone health classes `NOMINAL/DEGRADED/CRITICAL/FAILED`
Low battery	battery not above reserve; `notify-insufficient-battery`, `initiate-low-battery-return`	at 20 % swarm paused, operator chooses emergency landing or press on; at 0 % drone lost

Chapter 8

Verification and Results

8.1 Method

Each plan was verified in two ways. First, ENHSP was run directly on problems produced by the real expand, retarget and solve pipeline, for a range of drone counts. Second, the resulting routes were flown headless through the real drone threads and simulated firmware, and the spacing, altitudes and landing positions were measured. Negative tests were added where a plan must refuse a problem. The run logs in `plans/*/runs/` record the in-app requests.

8.2 In-App Planning Requests

Table 8.1 summarises the logged requests for the four longest-running plans. Every unsolvable travel and V request was traced to an infeasible combination of source, destination and no-fly area, where no safe path existed. The grid failures all occurred in a short window before a template fix (Section 8.7); every grid request after the fix solved.

Table 8.1: Logged planning requests per plan

Plan	Runs	Solved	Unsolvable	Notes
Forest search	69	69	0	no failures on record
Travel	83	76	7	infeasible no-fly inputs during early testing
V formation	65	62	2	2 infeasible inputs; 1 run closed mid-plan
Grid	19	15	4	all 4 before the hover-altitude fix

8.3 Scalability of the Cube Formation

Table 8.2 and Figure 8.1 show planning time and plan length for the cube formation. Plan length grows linearly with the number of drones, about four actions per drone. Planning time stays below 4 s up to 36 drones and reaches 50 s for a full $4 \times 4 \times 4$ block of 64 drones. A launch-order audit of the 25-drone plan found no drone taking off before its layer opened and no layer opening before the one below had finished climbing.

Table 8.2: Cube formation: ENHSP results through the real pipeline

Drones	8	9	12	16	25	27	36	64
Shape	2×2×2	3×2×2	2×3×3	2×3×3	3×3×3	3×3×3	3×4×4	4×4×4
Time (s)	0.4	0.5	0.6	0.7	1.2	1.8	3.4	50.0
Plan length	42	47	58	74	111	119	155	268

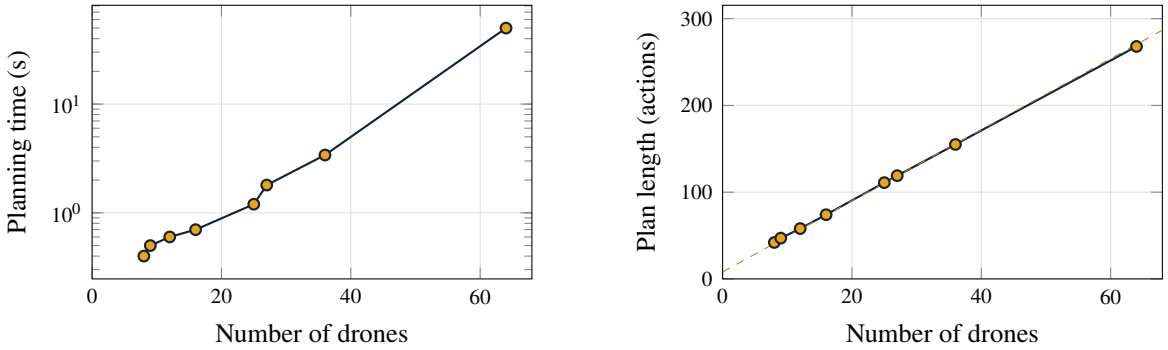


Figure 8.1: Cube formation scalability. Left: planning time (log scale). Right: plan length, close to linear in the number of drones (dashed line $\approx 4.1N + 8$).

8.4 Figure-Eight Capacity and Flight Quality

On an asymmetric eight with lobes of 337 m and 281 m, the computed capacity was 27 drones. ENHSP solved every count up to 27 and returned no plan for 28, in agreement with the Python check (Table 8.3). With a full 27-drone train, the minimum distance between any two drones at any waypoint step was 9.85 m, including at the crossing. Ten drones with 20 laps solved in 0.8 s, because laps are counted in PDDL and flown on board with a single repeat instruction rather than being unrolled.

Table 8.3: Figure-eight train: ENHSP results on an eight of capacity 27

Drones	2	5	12	25	27	28
Time (s)	0.4	0.4	0.7	2.7	3.3	–
Plan length	26	38	66	118	126	no plan (as intended)

After formation keeping moved on board (Chapter 6), a replay of a logged 16-drone, 2-lap run changed from a median neighbour gap of 23.7 m with a minimum of 0.0 m (collisions) to a median of 10.0 m, with 90 % of samples between 9.2 and 10.8 m, and every drone landed on its own pad.

8.5 Other Plans

Table 8.4 summarises the verification of the remaining plans.

Table 8.4: Verification summary of the remaining plans

Plan	Result
V formation	3-, 5- and 7-drone V's solve through the real pipeline after the neighbour chain was added.
Parabola	27-step plan with a detour, about 0.03–0.1 s. Negative checks (arm at 55 m, flat 50 m line, communication range 15 m) give no plan. In flight, altitudes held at 50/60/60 m, vertex–arm gap 14.1–15.0 m on a straight 1 km route.
Star show	6 drones × 1 lap 0.3 s; 40 drones × 5 laps 0.6 s. In flight (10 drones, 2 laps), worst drone 0.7–3.2 m off its slot while shapes were held, closest pair 9.3 m, every drone landed on its own pad.
Planets	2–8 drones × 3 laps, 0.2–0.3 s. With 8 drones every drone stayed within 0.4 m of its orbit and turned the correct direction and number of turns.
Road inspection	1.4 km and 3.8 km real roads followed with no measurable deviation from the road line; ENHSP 0.3 s; a source 393 m off the road was correctly refused.

8.6 A Complete Sample Plan

The parabola plan below, from run 2026-10-20_112051 with one detour location, shows the whole life cycle: pre-flight checks, take-offs and climbs, six link set-ups and three confirmations, slots, formation, two legs each followed by a shape hold, and completion.

ENHSP output — parabola formation with one detour (run 2026-10-20_112051)

```
  0.0: (pre-flight-check drone-vertex source)
  1.0: (takeoff drone-vertex source)
  2.0: (climb-to-cruise-altitude drone-vertex)
  3.0: (pre-flight-check drone-left source)
  4.0: (takeoff drone-left source)
  5.0: (climb-to-cruise-altitude drone-left)
  6.0: (pre-flight-check drone-right source)
  7.0: (takeoff drone-right source)
  8.0: (climb-to-cruise-altitude drone-right)
  9.0: (establish-comm-link drone-vertex drone-right)
 10.0: (establish-comm-link drone-vertex drone-left)
 11.0: (confirm-swarm-comm drone-vertex)
 12.0: (establish-comm-link drone-left drone-vertex)
 13.0: (establish-comm-link drone-left drone-right)
 14.0: (establish-comm-link drone-right drone-left)
 15.0: (establish-comm-link drone-right drone-vertex)
 16.0: (confirm-swarm-comm drone-right)
 17.0: (confirm-swarm-comm drone-left)
 18.0: (take-parabola-slot drone-vertex)
 19.0: (take-parabola-slot drone-left)
 20.0: (take-parabola-slot drone-right)
 21.0: (establish-parabola-formation drone-vertex source)
 22.0: (formation-cruise drone-vertex source detour1)
 23.0: (hold-parabola-shape drone-vertex detour1)
 24.0: (formation-cruise drone-vertex detour1 destination)
 25.0: (hold-parabola-shape drone-vertex destination)
 26.0: (complete-parabola-mission drone-vertex destination)
Plan-Length: 27   Planning Time: 31 ms   Expanded Nodes: 28   States Evaluated: 77
```

8.7 Defects Found and Fixed

Verification found several modelling defects. They are recorded here because each one taught a general lesson about writing PDDL for a template-expanding pipeline.

Table 8.5: Modelling defects found during verification

Plan	Defect	Fix and lesson
Grid	`hover-altitude` facts were written over three lines. The line-by-line cloner copied only the middle line, leaving the value undefined for every added drone; AIBR then reported any grid larger than 1×2 as unsolvable.	Rewritten as one line per fact. Templates must keep one fact per line.
Cube	Layer gate written as a comparison of two fluents. A 25-drone, 3-layer cube did not solve in 90 s.	Replaced by propositional layer objects; solved in 1.2 s. Prefer propositional gates for ordering.
Cube	A quantified variable `?l` in `establish-cube-formation` shadowed the action's location parameter, making every problem unsolvable.	Variable renamed. Never reuse a parameter name inside a quantifier.
V formation	Readiness checked only the ground link; there was no notion of which drones sit next to each other.	Added `wing-neighbor` chain and `neighbor-comm-ok`, with `detect-neighbor-link-loss`.
Figure-8	Ground-station correction caused oscillation and collisions in a 16-drone train.	Formation keeping moved on board with a shared pace (Chapter 6).

Chapter 9

Discussion and Limitations

9.1 What the Models Guarantee

A plan returned by ENHSP is a proof, within the model, that the mission can be completed without breaking any of the written rules: no drone flies with a failed flag, no move exceeds the battery, no formation forms without all its members in contact, the cube never climbs a layer through another, and the figure-eight never carries more drones than it can hold. Because these rules are written in readable PDDL, they can be reviewed directly, and a change to a rule takes effect in every mission that uses it.

9.2 Limitations

- **Symbolic communication.** The PDDL flags describe the assumed state of the links at planning time. They are not updated from live radio measurements during flight; run-time faults are handled by the ground station and the operator, not by replanning.
- **Faults are detected, not planned around.** The detection actions record a failure, but the goals still require healthy drones, so the planner does not produce contingency branches. Online replanning after a fault is left for future work.
- **Battery units.** Battery and energy in PDDL are model units with 100 as full. They are not calibrated against the 5 200 mAh run-time model, so the planning reserve (30 units) and the run-time low threshold (20 %) are separate safeguards.
- **Fault injection coverage.** The GUI offers GPS loss, communications loss and battery failure, but in the thread-based backend only GPS loss is currently delivered to the drones; communication loss is observed from missing messages and low battery arises from consumption.
- **Grounding cost.** Large two-dimensional grids are expensive to ground: a 16-drone (4×4) grid exceeded a 6 GB heap in local testing, even though larger cubes solve, because of the different structure of their quantified conditions.
- **Simulation only.** All results come from the simulator. No physical flight was performed, and the road inspection plan depends on an external routing service.

Chapter 10

Conclusion and Future Work

10.1 Conclusion

This thesis presented the PDDL planning layer of a drone swarm simulator. Ten mission plans were built on one shared vocabulary, covering travel, area search, four rigid formations, a figure-eight train, two shape-changing shows and road inspection. Communication was modelled as neighbour relations with per-drone flags, and, in the parabola plan, as an explicit mesh of range-checked links. Faults were modelled as detection actions with ground-controller responses, and battery as a numeric resource with a return reserve, complemented at run time by thresholds that pause the swarm and ask the operator.

The plans run through the real pipeline: they are retargeted to map inputs, expanded to the selected number of drones, solved by ENHSP and flown by the drone threads. The results show that careful modelling keeps planning fast for realistic swarm sizes: 64 drones in a cube in 50 s, a 27-drone figure-eight train in 3.3 s, and most missions in well under a second. The defects found during verification led to general modelling rules that will help anyone extending the plans.

10.2 Future Work

1. **Online replanning.** Feed run-time faults back into the PDDL state and replan the remaining mission, for example closing a formation around a lost drone.
2. **Calibrated energy model.** Derive `energy-required` and the reserve from the drone's battery capacity, current draw, speed and wind, so that planning and run-time thresholds use the same units.
3. **Live link model.** Update `neighbor-comm-ok` and `link-distance` from measured link quality, and extend the parabola plan's explicit link model to other formations.
4. **Complete fault injection.** Deliver communication loss and battery failure faults to the drone threads, matching the GUI options, and test the operator responses under each.
5. **Larger grids.** Reduce grounding cost for large two-dimensional grids, for example by the same propositional restructuring that made the cube scale.
6. **Hardware-in-the-loop tests.** Fly the plans against real flight firmware under emulation and, eventually, on physical drones.

References

[1] D. McDermott, M. Ghallab, A. Howe, C. Knoblock, A. Ram, M. Veloso, D. Weld and D. Wilkins, *PDDL – The Planning Domain Definition Language*, Technical Report CVC TR-98-003, Yale Center for Computational Vision and Control, 1998.

[2] M. Fox and D. Long, *PDDL2.1: An Extension to PDDL for Expressing Temporal Planning Domains*, Journal of Artificial Intelligence Research, vol. 20, pp. 61–124, 2003.

[3] M. Ghallab, D. Nau and P. Traverso, *Automated Planning: Theory and Practice*, Morgan Kaufmann, 2004.

[4] E. Scala, P. Haslum, S. Thiébaux and M. Ramirez, *Interval-Based Relaxation for General Numeric Planning*, Proceedings of the 22nd European Conference on Artificial Intelligence (ECAI), pp. 655–663, 2016.

[5] E. Scala, P. Haslum, S. Thiébaux and M. Ramirez, *Subgoaling Techniques for Satisficing and Optimal Numeric Planning*, Journal of Artificial Intelligence Research, vol. 68, pp. 691–752, 2020.

[6] S.-J. Chung, A. A. Paranjape, P. Dames, S. Shen and V. Kumar, *A Survey on Aerial Swarm Robotics*, IEEE Transactions on Robotics, vol. 34, no. 4, pp. 837–855, 2018.

[7] M. Brambilla, E. Ferrante, M. Birattari and M. Dorigo, *Swarm Robotics: A Review from the Swarm Engineering Perspective*, Swarm Intelligence, vol. 7, no. 1, pp. 1–41, 2013.

[8] C. W. Reynolds, *Flocks, Herds and Schools: A Distributed Behavioral Model*, Proceedings of SIGGRAPH '87, Computer Graphics, vol. 21, no. 4, pp. 25–34, 1987.

[9] L. E. Dubins, *On Curves of Minimal Length with a Constraint on Average Curvature, and with Prescribed Initial and Terminal Positions and Tangents*, American Journal of Mathematics, vol. 79, no. 3, pp. 497–516, 1957.

[10] D. Luxen and C. Vetter, *Real-Time Routing with OpenStreetMap Data*, Proceedings of the 19th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems, pp. 513–516, 2011.

[11] MAVLink Development Team, *MAVLink Micro Air Vehicle Communication Protocol*, `https://mavlink.io/en/`.

[12] E. Scala, *ENHSP – Expressive Numeric Heuristic Search Planner*, `https://sites.google.com/view/enhsp/`.

[13] Antmicro, *Renode: Open Source Simulation and Virtual Development Framework*, https://renode.io/.

[14] The Qt Company, *Qt for Python (PySide6)*, https://doc.qt.io/qtforpython-6/.

Appendix A

Complete PDDL Sources

This appendix lists two plans in full, as they appear in the project (blank lines removed). The point-to-point travel domain is the base of the shared vocabulary and contains the complete ground-controller fault handling. The parabola plan contains the explicit communication model. The sources of the other eight plans are in the project folder `plans/`; any of them can be added here with one `\lstinputlisting` line.

A.1 Point-to-Point Travel: Domain

```
(define (domain swarm-drone-mission)
    (:requirements
        :strips ;;Basic Planing Preconditions -> Action -> Effects
        :typing ;; Allows you to specify types drone, location, waypoint, controller
        :numeric-fluents ;; Allows numeric values such as: battery, energy-required, distance, health
        :negative-preconditions ;; Allows you to specify negative preconditions such as: (not (at ?d ?from))
        :conditional-effects ;; Allows you to specify conditional effects
    )
    ;; Types used in the mission
    (:types
        drone
        location
        waypoint
        controller
    )
    ;; Facts that describe the drone and mission state
    (:predicates
        ;; Drone position
        (at ?d - drone ?l - location)
        ;; Mission locations
        (source ?l - location)
        (destination ?l - location)
        ;; Waypoint information
        (waypoint-location ?w - waypoint ?l - location)
        ;; Route information
        (connected ?from - location ?to - location)
        (safe-route ?from - location ?to - location)
        (restricted-route ?from - location ?to - location)
        ;; Drone state
        (flying ?d - drone)
        (hovering ?d - drone)
        (mission-completed ?d - drone)
        ;; System health
        (gps-ok ?d - drone)
        (communication-ok ?d - drone)
        (drone-healthy ?d - drone)
        ;; Failure states
        (gps-lost ?d - drone)
        (communication-lost ?d - drone)
        (health-failure ?d - drone)
        ;; Controller commands and status
        (controller-notified ?d - drone)
        (controller-return ?d - drone)
```

```
        (controller-hover ?d - drone)
        (controller-sacrifice ?d - drone)
        ;; Warning messages
        (battery-insufficient ?d - drone)
        (health-warning ?d - drone)
        (gps-warning ?d - drone)
        (communication-warning ?d - drone)
        ;; Emergency states
        (emergency-return ?d - drone)
        (sacrificed ?d - drone)
        ;; Restricted area
        (no-fly-zone ?w - waypoint)
        ;; Mission checks
        (route-planned ?d - drone)
        (distance-checked ?d - drone)
        (battery-checked ?d - drone)
    )
    ;; Numeric values used by the planner
    (:functions
        ;; Battery information
        (battery ?d - drone)
        (max-battery ?d - drone)
        ;; Energy needed for a route
        (energy-required ?from - location ?to - location)
        ;; Distance between two locations
        (distance ?from - location ?to - location)
        ;; Drone health
        (health ?d - drone)
        (minimum-health ?d - drone)
        ;; Battery needed for emergency return
        (minimum-return-battery ?d - drone)
        ;; Battery used while hovering
        (hover-energy ?d - drone)
        ;; Energy used by the current route
        (route-energy ?d - drone)
        ;; Location coordinates
        (latitude ?l - location)
        (longitude ?l - location)
    )
    ;; Check the distance before planning the route
    (:action check-distance
        :parameters
            (?d - drone
             ?from - location
             ?to - location)
        :precondition
            (and
                (at ?d ?from)
                (connected ?from ?to)
            )
        :effect
            (distance-checked ?d)
    )
    ;; Check whether the drone has enough battery
    (:action check-battery
        :parameters
            (?d - drone
             ?from - location
             ?to - location)
        :precondition
            (and
                (at ?d ?from)
                (distance-checked ?d)
                (> (battery ?d)
                   (energy-required ?from ?to))
            )
        :effect
            (battery-checked ?d)
    )
    ;; Record that the battery is not enough
    (:action notify-insufficient-battery
        :parameters
            (?d - drone
```

```
            ?from - location
            ?to - location)
        :precondition
            (and
                (at ?d ?from)
                (distance-checked ?d)
                (<= (battery ?d)
                    (energy-required ?from ?to))
            )
        :effect
            (battery-insufficient ?d)
    )
    ;; Create a route only when all safety checks pass
    (:action plan-safe-route
        :parameters
            (?d - drone
             ?from - location
             ?to - location)
        :precondition
            (and
                (at ?d ?from)
                (connected ?from ?to)
                (safe-route ?from ?to)
                (battery-checked ?d)
                (gps-ok ?d)
                (communication-ok ?d)
                (drone-healthy ?d)
            )
        :effect
            (route-planned ?d)
    )
    ;; Move the drone and reduce its battery
    (:action travel
        :parameters
            (?d - drone
             ?from - location
             ?to - location)
        :precondition
            (and
                (at ?d ?from)
                (route-planned ?d)
                (safe-route ?from ?to)
                (gps-ok ?d)
                (communication-ok ?d)
                (drone-healthy ?d)
                (> (battery ?d)
                   (energy-required ?from ?to))
            )
        :effect
            (and
                ;; Move drone to the new location
                (at ?d ?to)
                (not (at ?d ?from))
                ;; Drone is now flying
                (flying ?d)
                ;; Reduce battery by route energy
                (decrease
                    (battery ?d)
                    (energy-required ?from ?to)
                )
                ;; Route must be planned again for the next move
                (not (route-planned ?d))
            )
    )
    ;; Complete the mission when the destination is reached
    (:action reach-destination
        :parameters
            (?d - drone
             ?dest - location)
        :precondition
            (and
                (at ?d ?dest)
                (destination ?dest)
```

```
            )
        :effect
            (and
                (mission-completed ?d)
                (not (flying ?d))
            )
    )
    ;; Detect a GPS failure during flight
    (:action detect-gps-loss
        :parameters
            (?d - drone)
        :precondition
            (and
                (flying ?d)
                (not (gps-ok ?d))
            )
        :effect
            (and
                (gps-lost ?d)
                (gps-warning ?d)
                (not (flying ?d))
            )
    )
    ;; Detect a communication failure during flight
    (:action detect-communication-loss
        :parameters
            (?d - drone)
        :precondition
            (and
                (flying ?d)
                (not (communication-ok ?d))
            )
        :effect
            (and
                (communication-lost ?d)
                (communication-warning ?d)
                (not (flying ?d))
            )
    )
    ;; Detect when drone health becomes too low
    (:action detect-health-failure
        :parameters
            (?d - drone)
        :precondition
            (and
                (flying ?d)
                (< (health ?d)
                   (minimum-health ?d))
            )
        :effect
            (and
                (health-failure ?d)
                (health-warning ?d)
                (not (flying ?d))
            )
    )
    ;; Inform the ground controller about a failure
    (:action notify-ground-controller
        :parameters
            (?d - drone
             ?c - controller)
        :precondition
            (or
                (gps-lost ?d)
                (communication-lost ?d)
                (health-failure ?d)
            )
        :effect
            (controller-notified ?d)
    )
    ;; Follow the controller's return command
    (:action controller-order-return
        :parameters
```

```
            (?d - drone)
        :precondition
            (and
                (controller-notified ?d)
                (controller-return ?d)
            )
        :effect
            (emergency-return ?d)
    )
    ;; Return the drone safely to the source
    (:action return-to-source
        :parameters
            (?d - drone
             ?current - location
             ?src - location)
        :precondition
            (and
                (at ?d ?current)
                (source ?src)
                (emergency-return ?d)
                (safe-route ?current ?src)
                (> (battery ?d)
                   (energy-required ?current ?src))
            )
        :effect
            (and
                (at ?d ?src)
                (not (at ?d ?current))
                (not (emergency-return ?d))
                (decrease
                    (battery ?d)
                    (energy-required ?current ?src)
                )
            )
    )
    ;; Follow the controller's hover command
    (:action controller-order-hover
        :parameters
            (?d - drone)
        :precondition
            (and
                (controller-notified ?d)
                (controller-hover ?d)
            )
        :effect
            (hovering ?d)
    )
    ;; Keep the drone hovering while enough battery remains
    (:action hover
        :parameters
            (?d - drone)
        :precondition
            (and
                (hovering ?d)
                (> (battery ?d)
                   (minimum-return-battery ?d))
            )
        :effect
            (decrease
                (battery ?d)
                (hover-energy ?d)
            )
    )
    ;; Start emergency return when battery becomes low
    (:action initiate-low-battery-return
        :parameters
            (?d - drone)
        :precondition
            (and
                (hovering ?d)
                (<= (battery ?d)
                    (minimum-return-battery ?d))
            )
```

```
        :effect
            (and
                (emergency-return ?d)
                (not (hovering ?d))
            )
    )
    ;; Emergency landing when the controller orders sacrifice
    (:action sacrifice-drone
        :parameters
            (?d - drone)
        :precondition
            (and
                (controller-notified ?d)
                (controller-sacrifice ?d)
            )
        :effect
            (and
                (sacrificed ?d)
                (not (flying ?d))
                (not (hovering ?d))
            )
    )
)
```

A.2 Point-to-Point Travel: Problem Template

```
(define (problem drone-mission-001)
    (:domain swarm-drone-mission)
    ;; Objects used in the mission
    (:objects
        drone1 - drone
        ground-controller - controller
        source - location
        destination - location
        waypoint1 - location
        waypoint2 - location
    )
    (:init
        ;; Drone starts at the source
        (at drone1 source)
        (source source)
        (destination destination)
        ;; Locations connected by possible routes
        (connected source destination)
        (connected source waypoint1)
        (connected waypoint1 waypoint2)
        (connected waypoint2 destination)
        (connected waypoint1 source)
        (connected waypoint2 waypoint1)
        (connected destination waypoint2)
        ;; Safe routes avoid the restricted direct route
        (safe-route source waypoint1)
        (safe-route waypoint1 waypoint2)
        (safe-route waypoint2 destination)
        (safe-route waypoint1 source)
        (safe-route waypoint2 waypoint1)
        (safe-route destination waypoint2)
        ;; Direct route is restricted
        (restricted-route source destination)
        ;; Drone starts in a normal operating state
        (gps-ok drone1)
        (communication-ok drone1)
        (drone-healthy drone1)
        ;; Geographic coordinates
        (= (latitude source) 13.0827)
        (= (longitude source) 80.2707)
        (= (latitude destination) 13.0674)
        (= (longitude destination) 80.2376)
```

```
        (= (latitude waypoint1) 13.0830)
        (= (longitude waypoint1) 80.2250)
        (= (latitude waypoint2) 13.0650)
        (= (longitude waypoint2) 80.2250)
        ;; Distances are provided by the mission-generation program
        (= (distance source destination) 4300)
        (= (distance source waypoint1) 3900)
        (= (distance waypoint1 waypoint2) 2100)
        (= (distance waypoint2 destination) 1800)
        (= (distance waypoint1 source) 3900)
        (= (distance waypoint2 waypoint1) 2100)
        (= (distance destination waypoint2) 1800)
        ;; Energy required for each route
        (= (energy-required source destination) 45)
        (= (energy-required source waypoint1) 20)
        (= (energy-required waypoint1 waypoint2) 12)
        (= (energy-required waypoint2 destination) 10)
        (= (energy-required waypoint1 source) 20)
        (= (energy-required waypoint2 waypoint1) 12)
        (= (energy-required destination waypoint2) 10)
        ;; Initial battery and battery limits
        (= (battery drone1) 85)
        (= (max-battery drone1) 100)
        ;; Start emergency return at or below 30% battery
        (= (minimum-return-battery drone1) 30)
        ;; Battery used for each hover action
        (= (hover-energy drone1) 2)
        ;; Initial drone health and minimum health
        (= (health drone1) 95)
        (= (minimum-health drone1) 40)
        ;; Waypoint 1 is inside the restricted area
        (no-fly-zone waypoint1)
    )
    (:goal
        (and
            ;; Mission is complete when the drone reaches the destination
            (mission-completed drone1)
            (at drone1 destination)
        )
    )
)
```

A.3 Parabola Formation: Domain

```
(define (domain parabola-formation-drone-mission)
    ;; Upward-parabola formation: three drones fly side by side on one line
    ;; across the direction of travel. The middle drone (the vertex) cruises
    ;; at 50 m and the two arm drones either side of it at 60 m, so seen
    ;; from behind the three make a "U" - an upward-opening parabola - and
    ;; fly it from the source to the destination. Every drone keeps a live
    ;; link with each of the other two the whole way
    (:requirements
        :strips
        :typing
        :equality
        :numeric-fluents
        :negative-preconditions
        :disjunctive-preconditions
        :universal-preconditions
        :conditional-effects
    )
    ;; Types used in the mission
    (:types
        drone
        location
        controller
    )
    ;; Facts that describe drone positions, states, and formation
```

```
    (:predicates
        ;; Drone position
        (at ?d - drone ?l - location)
        ;; Current location of the whole formation
        (formation-at ?l - location)
        ;; Mission locations and routes
        (source ?l - location)
        (destination ?l - location)
        (connected ?from - location ?to - location)
        (safe-route ?from - location ?to - location)
        ;; Flight states
        (landed ?d - drone)
        (airborne ?d - drone)
        (flying ?d - drone)
        (at-cruise-altitude ?d - drone)
        ;; Formation slots: the vertex in the middle (lowest point of the
        ;; parabola) and an arm drone either side of it on the same line
        (formation-leader ?d - drone)
        (slot-vertex ?d - drone)
        (arm-drone ?d - drone)
        (has-slot ?d - drone)
        (in-formation ?d - drone)
        ;; Formation status
        (parabola-established)
        (shape-held)
        ;; Communication between the drones. parabola-neighbor lists every
        ;; pair that must talk (all three pairs here - the two arms talk to
        ;; the vertex and to each other); link-up is a link that has been
        ;; set up and is in range; swarm-comm-ready means a drone has a
        ;; working link with every one of its neighbors
        (parabola-neighbor ?d1 - drone ?d2 - drone)
        (link-up ?d1 - drone ?d2 - drone)
        (neighbor-comm-ok ?d - drone)
        (swarm-comm-ready ?d - drone)
        ;; Pre-flight check
        (preflight-done ?d - drone)
        ;; System health
        (gps-ok ?d - drone)
        (communication-ok ?d - drone)
        (drone-healthy ?d - drone)
        ;; Failure states
        (gps-lost ?d - drone)
        (communication-lost ?d - drone)
        (health-failure ?d - drone)
        (controller-notified ?d - drone)
        ;; Mission status
        (mission-completed ?d - drone)
        (parabola-mission-complete)
    )
    ;; Numeric values used by the planner
    (:functions
        ;; Battery
        (battery ?d - drone)
        (max-battery ?d - drone)
        (minimum-return-battery ?d - drone)
        ;; Drone altitude
        (altitude ?d - drone)
        (hover-altitude ?d - drone)
        (target-altitude ?d - drone)
        ;; Energy consumption
        (climb-energy ?d - drone)
        (cruise-leg-energy ?d - drone)
        (energy-required ?from - location ?to - location)
        ;; Distance between locations
        (distance ?from - location ?to - location)
        ;; Drone health
        (health ?d - drone)
        (minimum-health ?d - drone)
        ;; Location coordinates
        (latitude ?l - location)
        (longitude ?l - location)
        ;; Parabola slot offsets: metres behind the vertex (negative =
        ;; behind) and metres to its right (negative = left). Both arms sit
```

```
        ;; level with the vertex, on the same line across the route
        (slot-along-offset ?d - drone)
        (slot-cross-offset ?d - drone)
        ;; Parabola shape: how much higher each arm flies than the vertex
        ;; (it must be exactly this, the same for both arms, so the shape is
        ;; a symmetric "U"), and the smallest 3-D gap any two drones may have
        (arm-rise)
        (min-separation)
        ;; Communication: straight-line distance between two drones in the
        ;; formation, and how far a drone-to-drone link can reach
        (link-distance ?d1 - drone ?d2 - drone)
        (comm-range)
    )
    ;; Check the drone before takeoff
    (:action pre-flight-check
        :parameters
            (?d - drone
             ?l - location)
        :precondition
            (and
                (at ?d ?l)
                (source ?l)
                (landed ?d)
                (has-slot ?d)
                (gps-ok ?d)
                (communication-ok ?d)
                (drone-healthy ?d)
                (>= (health ?d) (minimum-health ?d))
                (> (battery ?d) (minimum-return-battery ?d))
            )
        :effect
            (preflight-done ?d)
    )
    ;; Take off from the drone's own pad at the source - the pads are
    ;; already on the formation's line, one slot spacing apart, so the
    ;; three drones can lift off together without crossing
    (:action takeoff
        :parameters
            (?d - drone
             ?l - location)
        :precondition
            (and
                (at ?d ?l)
                (source ?l)
                (landed ?d)
                (preflight-done ?d)
                (gps-ok ?d)
                (communication-ok ?d)
                (drone-healthy ?d)
            )
        :effect
            (and
                (not (landed ?d))
                (airborne ?d)
                (flying ?d)
                (assign (altitude ?d) (hover-altitude ?d))
            )
    )
    ;; Climb from hover height to the drone's own cruise altitude - 50 m for
    ;; the vertex, 60 m for each arm
    (:action climb-to-cruise-altitude
        :parameters
            (?d - drone)
        :precondition
            (and
                (airborne ?d)
                (not (at-cruise-altitude ?d))
                (< (altitude ?d) (target-altitude ?d))
                (gps-ok ?d)
                (communication-ok ?d)
                (drone-healthy ?d)
                (> (battery ?d) (climb-energy ?d))
            )
```

```
        :effect
            (and
                (assign (altitude ?d) (target-altitude ?d))
                (at-cruise-altitude ?d)
                (decrease (battery ?d) (climb-energy ?d))
            )
    )
    ;; Set up the link from one drone to a neighbor - both must be up at
    ;; cruise altitude with their radios working, and close enough for the
    ;; link to reach
    (:action establish-comm-link
        :parameters
            (?d1 - drone
             ?d2 - drone)
        :precondition
            (and
                (parabola-neighbor ?d1 ?d2)
                (not (link-up ?d1 ?d2))
                (airborne ?d1)
                (airborne ?d2)
                (at-cruise-altitude ?d1)
                (at-cruise-altitude ?d2)
                (communication-ok ?d1)
                (communication-ok ?d2)
                (neighbor-comm-ok ?d1)
                (neighbor-comm-ok ?d2)
                (<= (link-distance ?d1 ?d2) (comm-range))
            )
        :effect
            (link-up ?d1 ?d2)
    )
    ;; A drone is ready to fly in the swarm once it has a link to every one
    ;; of its neighbors
    (:action confirm-swarm-comm
        :parameters
            (?d - drone)
        :precondition
            (and
                (airborne ?d)
                (not (swarm-comm-ready ?d))
                (communication-ok ?d)
                (neighbor-comm-ok ?d)
                (forall (?n - drone)
                    (or
                        (not (parabola-neighbor ?d ?n))
                        (link-up ?d ?n)
                    )
                )
            )
        :effect
            (swarm-comm-ready ?d)
    )
    ;; Move the drone into its assigned parabola slot
    (:action take-parabola-slot
        :parameters
            (?d - drone)
        :precondition
            (and
                (airborne ?d)
                (has-slot ?d)
                (at-cruise-altitude ?d)
                (>= (altitude ?d) (target-altitude ?d))
                (not (in-formation ?d))
                (swarm-comm-ready ?d)
                (gps-ok ?d)
                (communication-ok ?d)
                (drone-healthy ?d)
            )
        :effect
            (in-formation ?d)
    )
    ;; Establish the parabola when all three drones are in their slots and
    ;; talking to each other. The shape must open upward: each arm flies
```

```
    ;; exactly arm-rise above the vertex (so both arms are level with each
    ;; other), and no two drones are closer than the minimum separation
    (:action establish-parabola-formation
        :parameters
            (?vtx - drone
             ?l - location)
        :precondition
            (and
                (formation-leader ?vtx)
                (slot-vertex ?vtx)
                (in-formation ?vtx)
                (at ?vtx ?l)
                (at-cruise-altitude ?vtx)
                (>= (altitude ?vtx) (target-altitude ?vtx))
                (swarm-comm-ready ?vtx)
                (forall (?a - drone)
                    (or
                        (not (arm-drone ?a))
                        (and
                            (in-formation ?a)
                            (at ?a ?l)
                            (at-cruise-altitude ?a)
                            (>= (altitude ?a) (target-altitude ?a))
                            (= (- (target-altitude ?a) (target-altitude ?vtx)) (arm-rise))
                            (>= (link-distance ?vtx ?a) (min-separation))
                            (swarm-comm-ready ?a)
                        )
                    )
                )
            )
        :effect
            (and
                (parabola-established)
                (formation-at ?l)
                (shape-held)
            )
    )
    ;; Move the vertex to the next location - the arms hold their slots
    ;; either side of it
    (:action formation-cruise
        :parameters
            (?vtx - drone
             ?from - location
             ?to - location)
        :precondition
            (and
                (parabola-established)
                (shape-held)
                (formation-leader ?vtx)
                (slot-vertex ?vtx)
                (in-formation ?vtx)
                (formation-at ?from)
                (at ?vtx ?from)
                (connected ?from ?to)
                (safe-route ?from ?to)
                (at-cruise-altitude ?vtx)
                (>= (altitude ?vtx) (target-altitude ?vtx))
                (gps-ok ?vtx)
                (communication-ok ?vtx)
                (drone-healthy ?vtx)
                (neighbor-comm-ok ?vtx)
                (swarm-comm-ready ?vtx)
                (> (battery ?vtx) (energy-required ?from ?to))
            )
        :effect
            (and
                (not (formation-at ?from))
                (formation-at ?to)
                (not (at ?vtx ?from))
                (at ?vtx ?to)
                ;; The arms must re-form the parabola before the next leg
                (not (shape-held))
                (decrease (battery ?vtx) (energy-required ?from ?to))
```

```
            )
    )
    ;; Bring both arms level with the vertex again. Only possible while
    ;; every link in the formation is still up - the arms hold their slots
    ;; from the positions they share with the vertex and with each other
    (:action hold-parabola-shape
        :parameters
            (?vtx - drone
             ?to - location)
        :precondition
            (and
                (parabola-established)
                (not (shape-held))
                (formation-leader ?vtx)
                (in-formation ?vtx)
                (formation-at ?to)
                (at ?vtx ?to)
                (forall (?d1 - drone)
                    (forall (?d2 - drone)
                        (or
                            (not (parabola-neighbor ?d1 ?d2))
                            (link-up ?d1 ?d2)
                        )
                    )
                )
                (forall (?a - drone)
                    (or
                        (not (arm-drone ?a))
                        (and
                            (in-formation ?a)
                            (at-cruise-altitude ?a)
                            (>= (altitude ?a) (target-altitude ?a))
                            (gps-ok ?a)
                            (communication-ok ?a)
                            (drone-healthy ?a)
                            (neighbor-comm-ok ?a)
                            (swarm-comm-ready ?a)
                            (> (battery ?a) (cruise-leg-energy ?a))
                        )
                    )
                )
            )
        :effect
            (and
                (forall (?a - drone) (forall (?x - location)
                    (when (and (arm-drone ?a) (at ?a ?x))
                        (not (at ?a ?x)))))
                (forall (?a - drone)
                    (when (arm-drone ?a) (at ?a ?to)))
                (forall (?a - drone)
                    (when (arm-drone ?a)
                        (decrease (battery ?a) (cruise-leg-energy ?a))))
                (shape-held)
            )
    )
    ;; Complete the mission when the whole parabola reaches the destination
    (:action complete-parabola-mission
        :parameters
            (?vtx - drone
             ?dest - location)
        :precondition
            (and
                (destination ?dest)
                (parabola-established)
                (shape-held)
                (formation-at ?dest)
                (formation-leader ?vtx)
                (slot-vertex ?vtx)
                (in-formation ?vtx)
                (at ?vtx ?dest)
                (forall (?a - drone)
                    (or
                        (not (arm-drone ?a))
```

```
                        (and
                            (in-formation ?a)
                            (at ?a ?dest)
                        )
                    )
                )
            )
        :effect
            (and
                (mission-completed ?vtx)
                (forall (?a - drone)
                    (when (arm-drone ?a)
                        (mission-completed ?a)))
                (parabola-mission-complete)
            )
    )
    ;; Detect GPS failure
    (:action detect-gps-loss
        :parameters
            (?d - drone)
        :precondition
            (and
                (flying ?d)
                (not (gps-ok ?d))
            )
        :effect
            (and
                (gps-lost ?d)
                (not (flying ?d))
                (not (in-formation ?d))
            )
    )
    ;; Detect communication failure
    (:action detect-communication-loss
        :parameters
            (?d - drone)
        :precondition
            (and
                (flying ?d)
                (not (communication-ok ?d))
            )
        :effect
            (and
                (communication-lost ?d)
                (not (flying ?d))
                (not (in-formation ?d))
                (not (swarm-comm-ready ?d))
            )
    )
    ;; Detect loss of the drone-to-drone links - every link the drone has
    ;; drops with it, so the shape can't be re-formed until it is restored
    (:action detect-neighbor-link-loss
        :parameters
            (?d - drone)
        :precondition
            (and
                (flying ?d)
                (not (neighbor-comm-ok ?d))
            )
        :effect
            (and
                (communication-lost ?d)
                (not (flying ?d))
                (not (in-formation ?d))
                (not (swarm-comm-ready ?d))
                (forall (?n - drone)
                    (when (parabola-neighbor ?d ?n)
                        (and
                            (not (link-up ?d ?n))
                            (not (link-up ?n ?d))
                        )
                    )
                )
```

```
            )
    )
    ;; Detect when drone health becomes too low
    (:action detect-health-failure
        :parameters
            (?d - drone)
        :precondition
            (and
                (flying ?d)
                (< (health ?d) (minimum-health ?d))
            )
        :effect
            (and
                (health-failure ?d)
                (not (flying ?d))
                (not (in-formation ?d))
            )
    )
    ;; Inform the ground controller about a failure
    (:action notify-ground-controller
        :parameters
            (?d - drone
             ?c - controller)
        :precondition
            (and
                (or
                    (gps-lost ?d)
                    (communication-lost ?d)
                    (health-failure ?d)
                )
            )
        :effect
            (controller-notified ?d)
    )
)
```

A.4 Parabola Formation: Problem Template

```
(define (problem parabola-formation-mission-001)
    (:domain parabola-formation-drone-mission)
    ;; Objects used in the parabola mission
    (:objects
        drone-vertex - drone
        drone-left   - drone
        drone-right  - drone
        ground-controller - controller
        source       - location
        destination  - location
    )
    (:init
        ;; All drones start landed at the source
        (at drone-vertex source)
        (at drone-left   source)
        (at drone-right  source)
        (landed drone-vertex)
        (landed drone-left)
        (landed drone-right)
        (= (altitude drone-vertex) 0)
        (= (altitude drone-left)   0)
        (= (altitude drone-right)  0)
        (source source)
        (destination destination)
        ;; Direct route between source and destination
        (connected source destination)
        (connected destination source)
        (safe-route source destination)
        (safe-route destination source)
        ;; The vertex leads from the middle; one arm drone either side
```

```
        (formation-leader drone-vertex)
        (slot-vertex drone-vertex)
        (arm-drone drone-left)
        (arm-drone drone-right)
        (has-slot drone-vertex)
        (has-slot drone-left)
        (has-slot drone-right)
        ;; All three on one line across the route, 10 m apart
        (= (slot-along-offset drone-vertex) 0)
        (= (slot-cross-offset drone-vertex) 0)
        (= (slot-along-offset drone-left)   0)
        (= (slot-cross-offset drone-left)  -10)
        (= (slot-along-offset drone-right)  0)
        (= (slot-cross-offset drone-right)  10)
        ;; Parabola shape - each arm 10 m above the vertex (50 m -> 60 m),
        ;; no two drones closer than 10 m
        (= (arm-rise) 10)
        (= (min-separation) 10)
        ;; Communication - every drone talks to both of the others: each
        ;; arm to the vertex, and the two arms to each other
        (parabola-neighbor drone-vertex drone-left)
        (parabola-neighbor drone-left drone-vertex)
        (parabola-neighbor drone-vertex drone-right)
        (parabola-neighbor drone-right drone-vertex)
        (parabola-neighbor drone-left drone-right)
        (parabola-neighbor drone-right drone-left)
        (neighbor-comm-ok drone-vertex)
        (neighbor-comm-ok drone-left)
        (neighbor-comm-ok drone-right)
        ;; Straight-line gaps in the formation: vertex to an arm is 10 m
        ;; across and 10 m up, arm to arm is 20 m across at the same height
        (= (link-distance drone-vertex drone-left)  14.142136)
        (= (link-distance drone-left drone-vertex)  14.142136)
        (= (link-distance drone-vertex drone-right) 14.142136)
        (= (link-distance drone-right drone-vertex) 14.142136)
        (= (link-distance drone-left drone-right)   20)
        (= (link-distance drone-right drone-left)   20)
        ;; Drone-to-drone link range
        (= (comm-range) 100)
        ;; All drones start with normal health and system status
        (gps-ok drone-vertex)
        (communication-ok drone-vertex)
        (drone-healthy drone-vertex)
        (gps-ok drone-left)
        (communication-ok drone-left)
        (drone-healthy drone-left)
        (gps-ok drone-right)
        (communication-ok drone-right)
        (drone-healthy drone-right)
        (= (health drone-vertex) 100)
        (= (health drone-left)   100)
        (= (health drone-right)  100)
        (= (minimum-health drone-vertex) 40)
        (= (minimum-health drone-left)   40)
        (= (minimum-health drone-right)  40)
        ;; Initial battery
        (= (battery drone-vertex) 100)
        (= (battery drone-left)   100)
        (= (battery drone-right)  100)
        (= (max-battery drone-vertex) 100)
        (= (max-battery drone-left)   100)
        (= (max-battery drone-right)  100)
        ;; Battery level for emergency return
        (= (minimum-return-battery drone-vertex) 30)
        (= (minimum-return-battery drone-left)   30)
        (= (minimum-return-battery drone-right)  30)
        ;; Drones start at 2 m; the vertex climbs to 50 m, the arms to 60 m
        (= (hover-altitude drone-vertex) 2)
        (= (hover-altitude drone-left)   2)
        (= (hover-altitude drone-right)  2)
        (= (target-altitude drone-vertex) 50)
        (= (target-altitude drone-left)   60)
        (= (target-altitude drone-right)  60)
```

```
        ;; Battery used during the climb (the arms climb 10 m further)
        (= (climb-energy drone-vertex) 4)
        (= (climb-energy drone-left)   5)
        (= (climb-energy drone-right)  5)
        ;; Battery used by each drone for one cruise leg
        (= (cruise-leg-energy drone-vertex) 12)
        (= (cruise-leg-energy drone-left)   12)
        (= (cruise-leg-energy drone-right)  12)
        ;; Source and destination coordinates
        (= (latitude source)       18.3901547)
        (= (longitude source)      79.0495640)
        (= (latitude destination)  18.3876026)
        (= (longitude destination) 79.0816646)
        ;; Direct route distance and energy cost
        (= (distance source destination) 3399.03)
        (= (distance destination source) 3399.03)
        (= (energy-required source destination) 10.00)
        (= (energy-required destination source) 10.00)
    )
    (:goal
        (and
            ;; Parabola is established
            (parabola-established)
            (in-formation drone-vertex)
            (in-formation drone-left)
            (in-formation drone-right)
            ;; Every drone has a working link with both of the others
            (swarm-comm-ready drone-vertex)
            (swarm-comm-ready drone-left)
            (swarm-comm-ready drone-right)
            ;; Vertex at 50 m, both arms at 60 m
            (at-cruise-altitude drone-vertex)
            (at-cruise-altitude drone-left)
            (at-cruise-altitude drone-right)
            (>= (altitude drone-vertex) 50)
            (>= (altitude drone-left)   60)
            (>= (altitude drone-right)  60)
            ;; Formation reached the destination
            (formation-at destination)
            (at drone-vertex destination)
            (at drone-left   destination)
            (at drone-right  destination)
            ;; Mission is completed
            (parabola-mission-complete)
            (mission-completed drone-vertex)
            (mission-completed drone-left)
            (mission-completed drone-right)
        )
    )
)
```